

Advanced Computer Systems (CIS 570)

Instruction Set Architecture

Slides are adopted from:

- Patterson and Hennessy, Computer Architecture a Quantitative Approach (Fifth edition)
- Notes by Dr. O. Mutlu, CMU
- Patterson and Hennessy, Computer Organization and Design: The Hardware/Software Interface, Fourth Edition
- Notes by Dr. M. Geiger, UMA.
- Vincent Heuring and Harry Jordan, Computer Systems design and Architecture, Second Edition

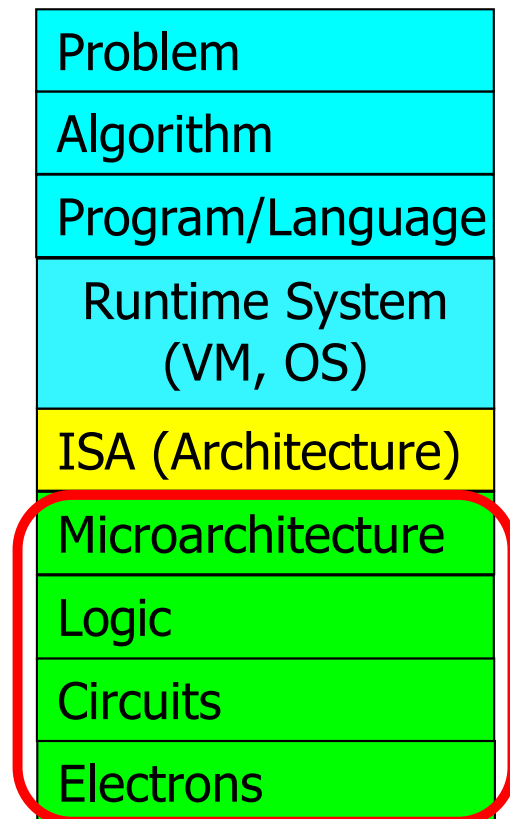
What is Computing?

- Solving a problem using electrons

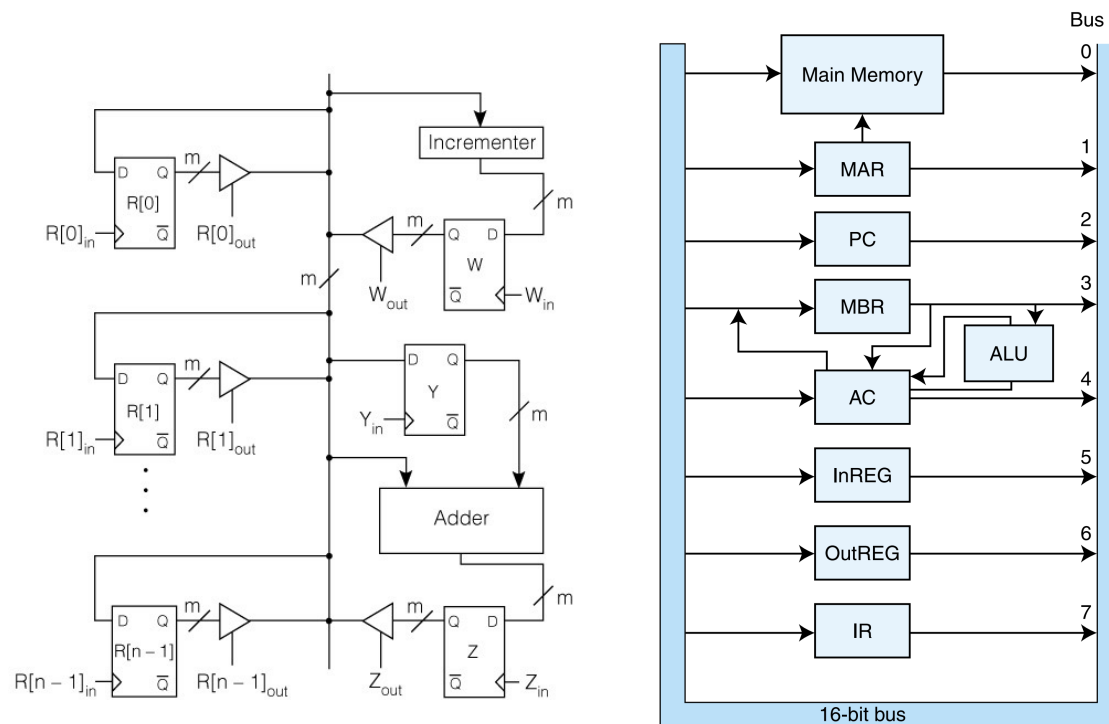
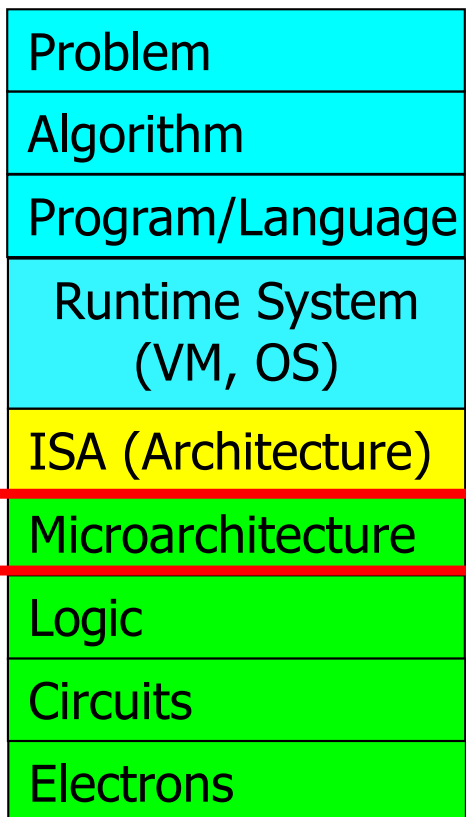
Problem
Algorithm
Program/Language
Runtime System (VM, OS)
ISA (Architecture)
Microarchitecture
Logic
Circuits
Electrons

- Coordination of many levels of abstraction

Hardware Stack

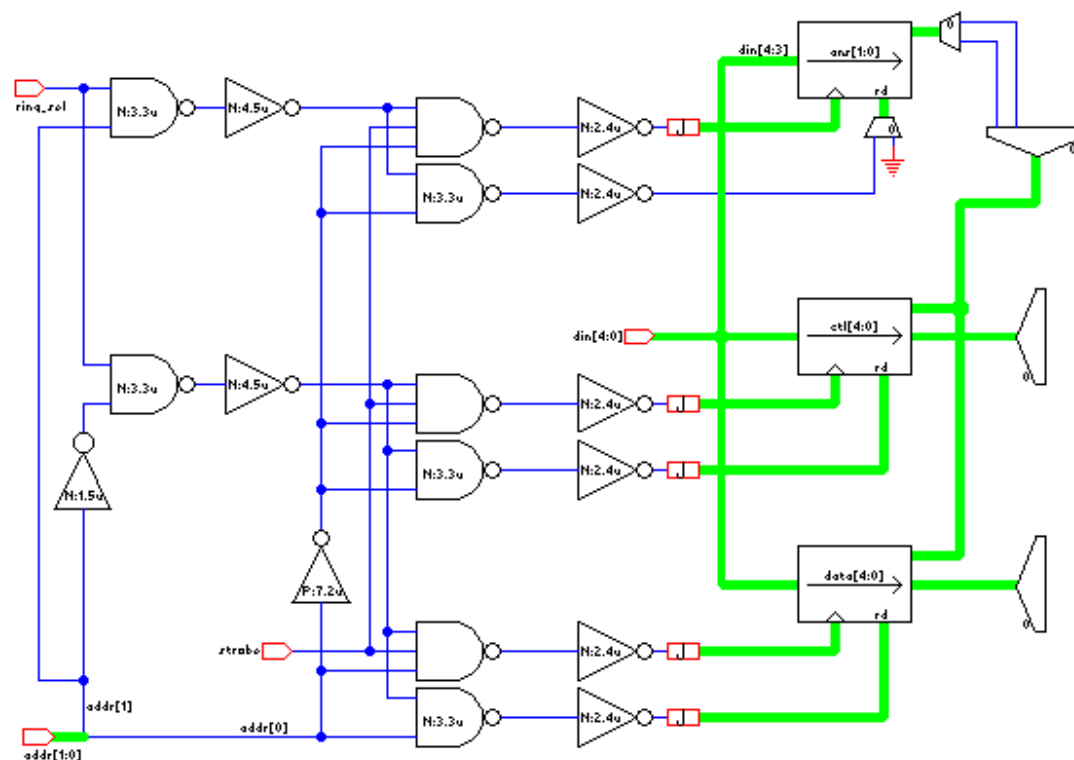
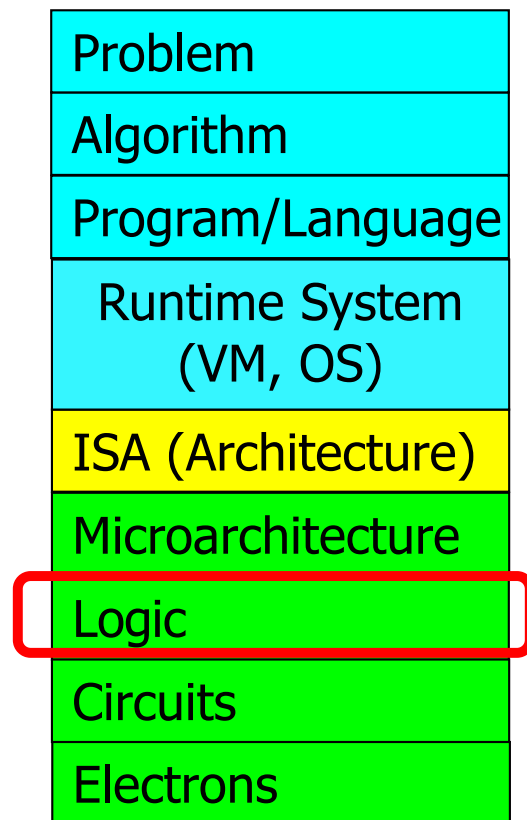


Hardware Stack



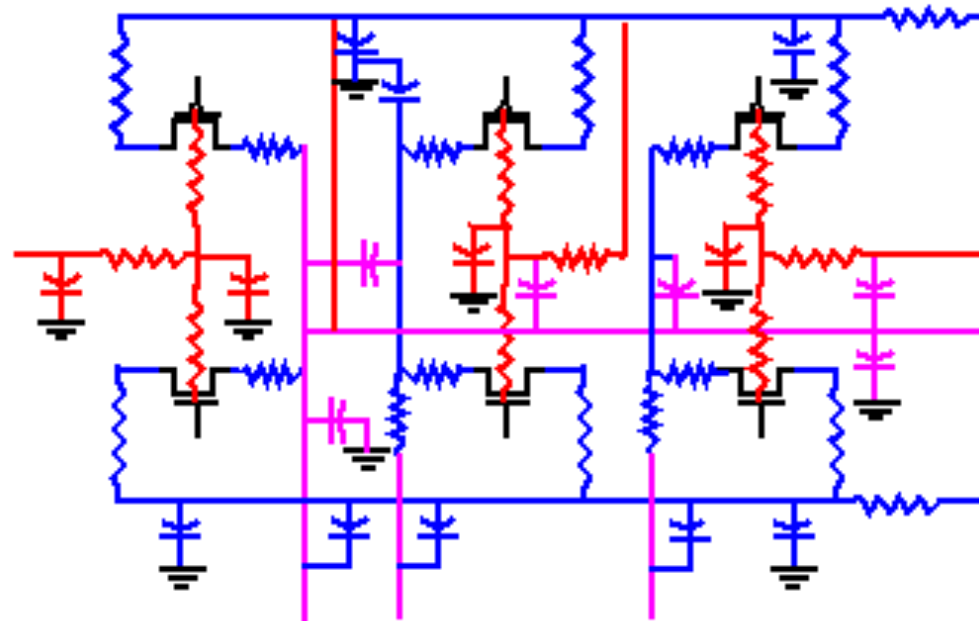
Data & Control Paths

Hardware Stack



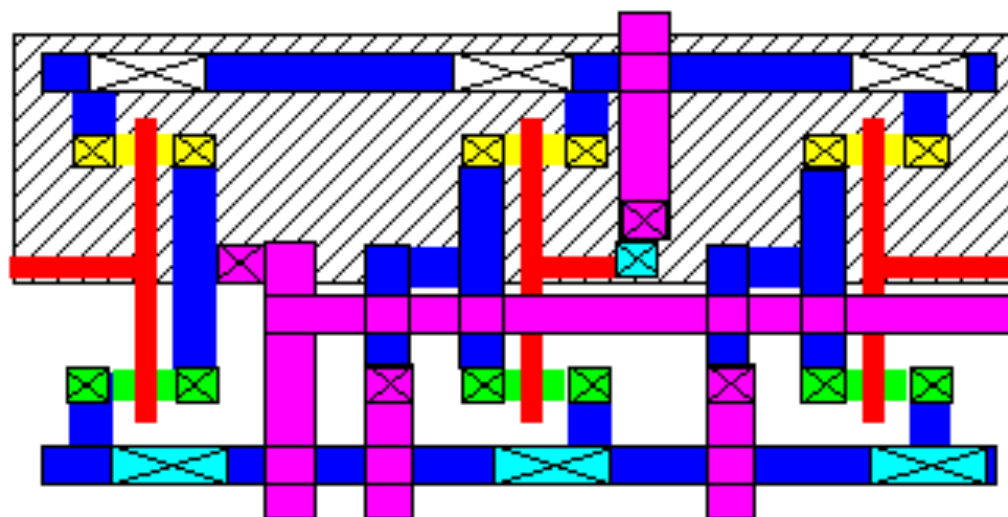
Hardware Stack

Problem
Algorithm
Program/Language
Runtime System (VM, OS)
ISA (Architecture)
Microarchitecture
Logic
Circuits
Electrons



Hardware Stack

Problem
Algorithm
Program/Language
Runtime System (VM, OS)
ISA (Architecture)
Microarchitecture
Logic
Circuits
Electrons



Hardware Hierarchy Abstraction

Micro level:

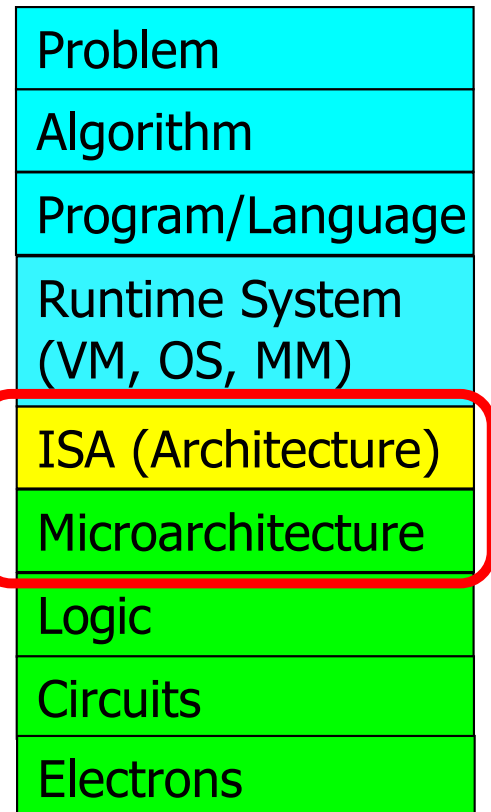
- Transistors;
- Logic gates, memory cells, special circuits;
- Single-bit adders, multiplexers, decoders, flip-flops;
- Word-wide adders, multiplexers, decoders, registers, buses;
- ALUs, barrel shifters, register banks, memory blocks;
- Processor, cache and memory management organizations;

Macro Level:

- Processors, peripheral cells, cache memories, memory management units;
- Integrated system chips;
- Printed circuit boards;
- Mobile telephones, PCs, engine controllers.

ISA vs. Microarchitecture

- **ISA**: Specifies how the **programmer sees program** (instructions to be executed)
 - Execution is as a **sequence of instructions** that are **executed in an order** (semantics)
- **Microarchitecture**: specifies **how** the underlying **implementation actually executes** instructions
 - Can **execute instructions** in **any order** as long as it **obeys the semantics** specified by the ISA when instruction results are visible to software (**programmer sees the right order**)



ISA vs. Microarchitecture (cont.)

- All major **instruction set architectures** today **use Von Neumann model** (e.g. RISC-V, IA-32, IA-64, x86-32, x86-64, ARM, MIPS, SPARC, POWER)
- **Underneath** (at the **microarchitecture** level), the **execution model** of almost **all implementations** (or, microarchitectures) is **very different** (e.g. X86)
 - **Pipelined instruction execution**: Intel 80486 μ arch
 - **Multiple instructions at a time**: Intel Pentium μ arch
 - **Out-of-order execution**: Intel Pentium Pro μ arch
 - **Speculation**: Intel i9, i7, i5, i3 μ arch
 - **Separate** instruction and data **caches**
- Major **difference** between **ISA** and **microarchitecture**
 - What happens underneath (that is **not consistent with the Von Neumann model**) is **not exposed** to **software**

ISA vs. Microarchitecture (Example)

- What is part of ISA vs. Uarch?
 - Acceleration pedal: interface for “acceleration”
 - Internals of the engine: implement “acceleration”
- Implementation (μ arch) can be different as long as it satisfies the specification (ISA)
 - Add instruction vs. Adder implementation
 - Bit serial, ripple carry, carry lookahead adders are all part of microarchitecture
 - x86 ISA has many implementations: 286, 386, 486, Pentium, Pentium Pro, Pentium 4, i3,i5,i7,i9, AMD Ryzen..
- Microarchitecture usually changes faster than ISA
 - Few ISAs (IA-32, x86-64, ARM, SPARC, MIPS, Alpha) but many μ archs
 - Why? Software will break (no backward compatibility)

ISA vs. Microarchitecture (cont.)

- ISA

- Agreed upon **interface between software and hardware**
 - SW/compiler assumes, HW promises
- What the **software writer needs to know** to write and debug system/user programs

- Microarchitecture

- Specific **implementation** of an ISA
- **Not visible** to the software writer

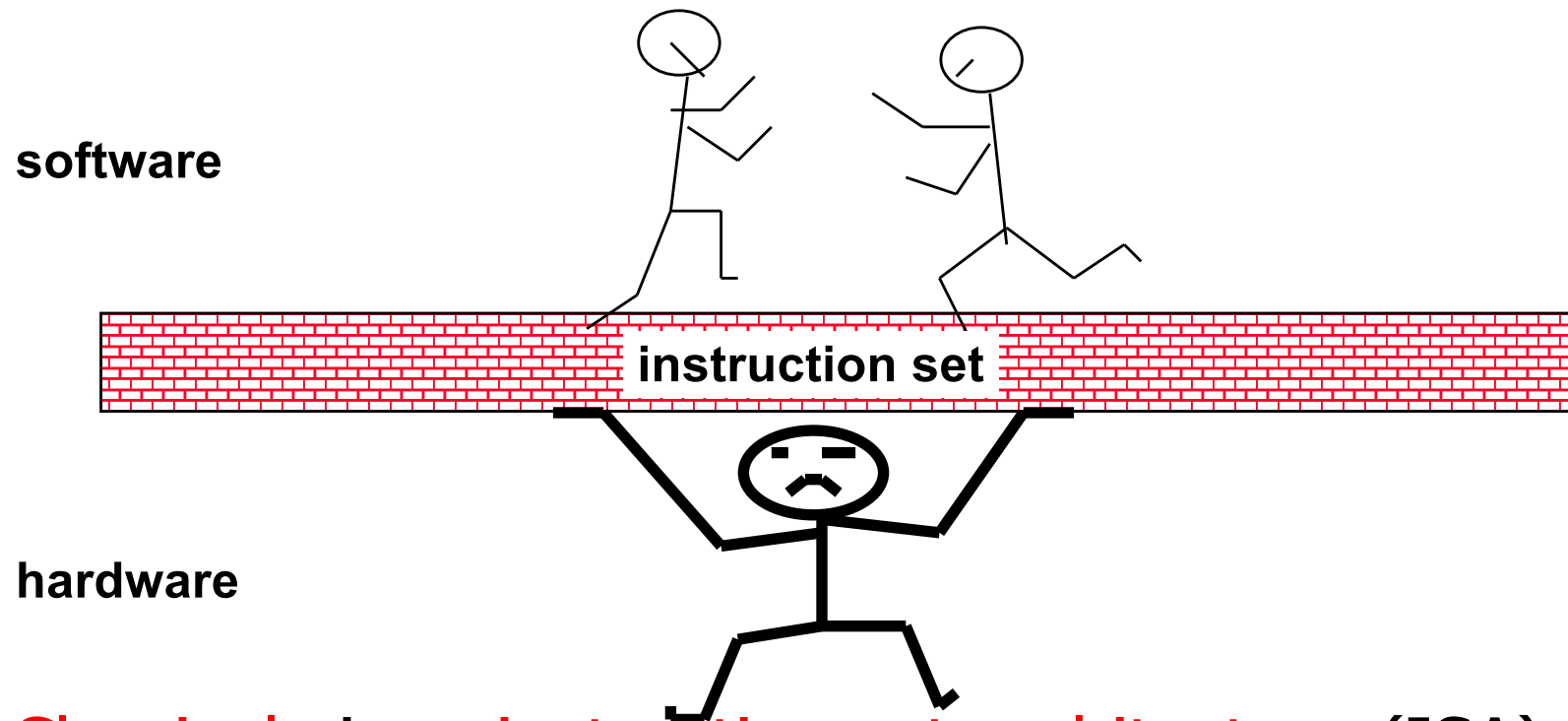
- Microprocessor

- ISA, μ arch, circuits
- “**Architecture**” = **ISA** + **microarchitecture**

Property of ISA vs. Uarch?

- HW supports an ADD instruction ISA
- ADD instruction's opcode ISA
- The exact implementation of MUL instruction Uarch
- Whether or not the machine employs pipelined instruction execution Uarch
- Number of general-purpose registers ISA

What is computer architecture?



- Classical view: instruction set architecture (ISA)
 - Boundary between hardware and software
 - Provides abstraction at both high level and low level
- More modern view: ISA + hardware design (μarch)
 - Can talk about processor architecture, system architecture

Examples of ISAs

- X86-64
- RISC ISAs: Alpha, MIPS, SPARC, ARM
- PDP-x: Programmed Data Processor (PDP-11)
- VAX (very complex, Virtual Address Extension by DEC)
- IBM 360
- CDC 6600
- SIMD ISAs: CRAY-1, Connection Machine (e.g. add on a million element)
- VLIW ISAs: Multiflow, Cydrome, IA-64 (EPIC), (e.g add, mult, load and store operations in one instruction)
- PowerPC, POWER
- What are the fundamental differences?
 - E.g., how instructions are specified and what they do
 - E.g., how complex are the instructions

Examples of μ Arch

- Anything done in hardware without exposure to software
 - Pipelining
 - In-order versus out-of-order instruction execution
 - Memory access scheduling policy
 - Speculative execution
 - Superscalar processing (multiple instruction issue?)
 - Clock gating
 - Caching? Levels, size, associativity, replacement policy
 - Prefetching?
 - Voltage/frequency scaling?
 - Error correction?

More on Execution Models

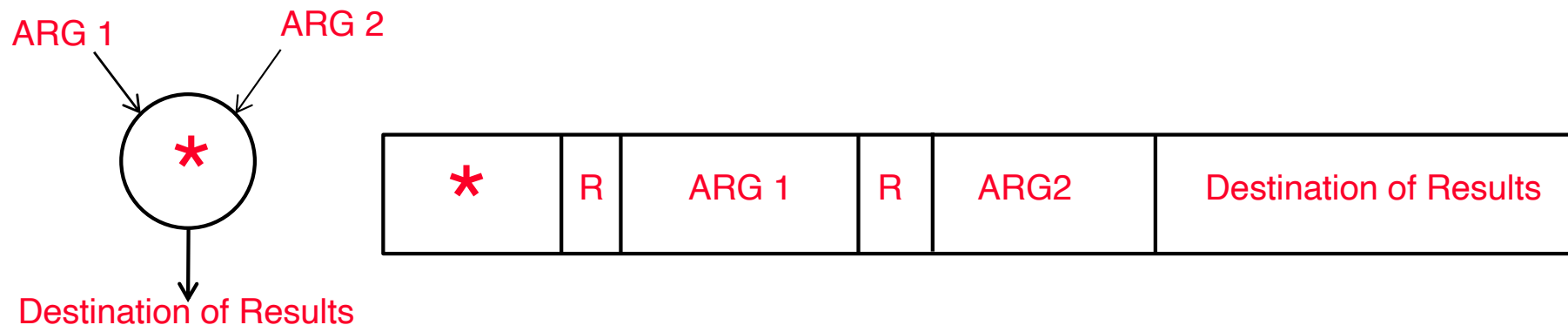
The Von Neumann Model/Architecture

- Called **stored program computer**
- An instruction is **fetch**ed and **executed** in **control flow** order:
- **Stored program**
 - Instructions stored in **unified linear memory** array which holds both **instructions and data** (interpretation of a stored value being instruction or data depends on the **control signals**)
- **Sequential instruction processing**
 - **One instruction** processed (**fetch**ed, **executed**, and **completed**) at a time controlled by **Program Counter** which is **advanced sequentially** except for **control transfer instructions**

More on Execution Models (cont.)

The **Dataflow** Model/Architecture

- An instruction is **fetch**ed and **executed** in **data flow order** (order of execution is **specified** by **data flow**)
 - Dataflow machine consists of **dataflow nodes** (**instructions**)
 - An Instruction specifies **who** receives the result and **fires** (fetched and executed) when **all operands** are available
 - **No PC** which makes it **inherently more parallel**

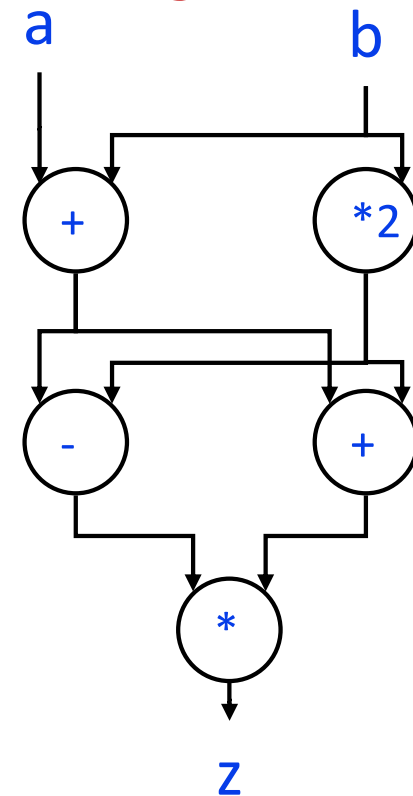


Von Neumann vs Dataflow

- Consider a Von Neumann program
 - What is the **significance** of the **program order**?
 - What is the **significance** of the **storage locations**?

```
v <= a + b;  
w <= b * 2;  
x <= v - w;  
y <= v + w;  
z <= x * y;
```

Sequential

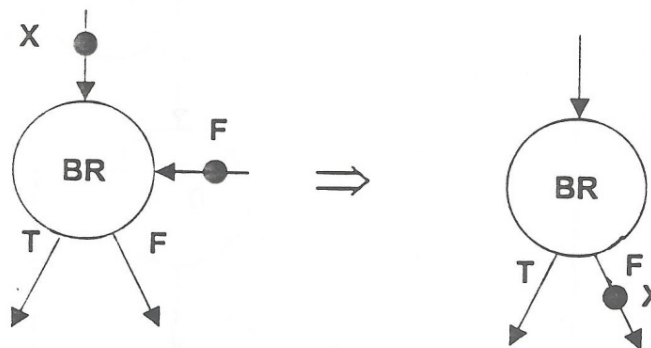


Dataflow

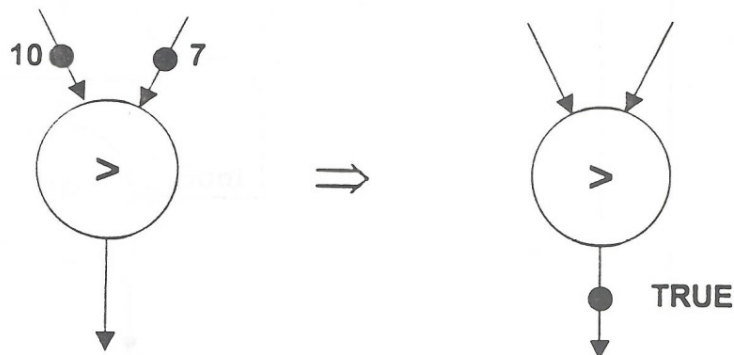
- Which model is more natural to you as a programmer?

Data Flow Nodes

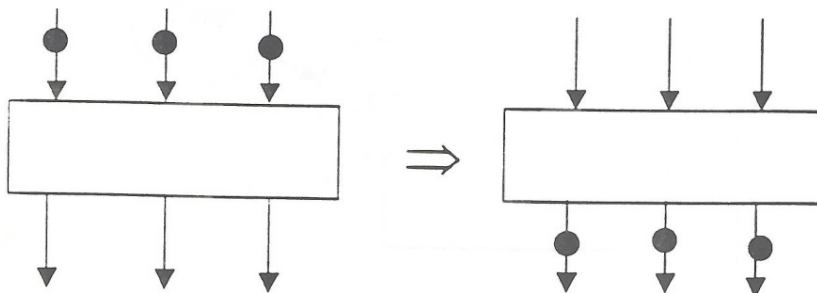
*Conditional



*Relational

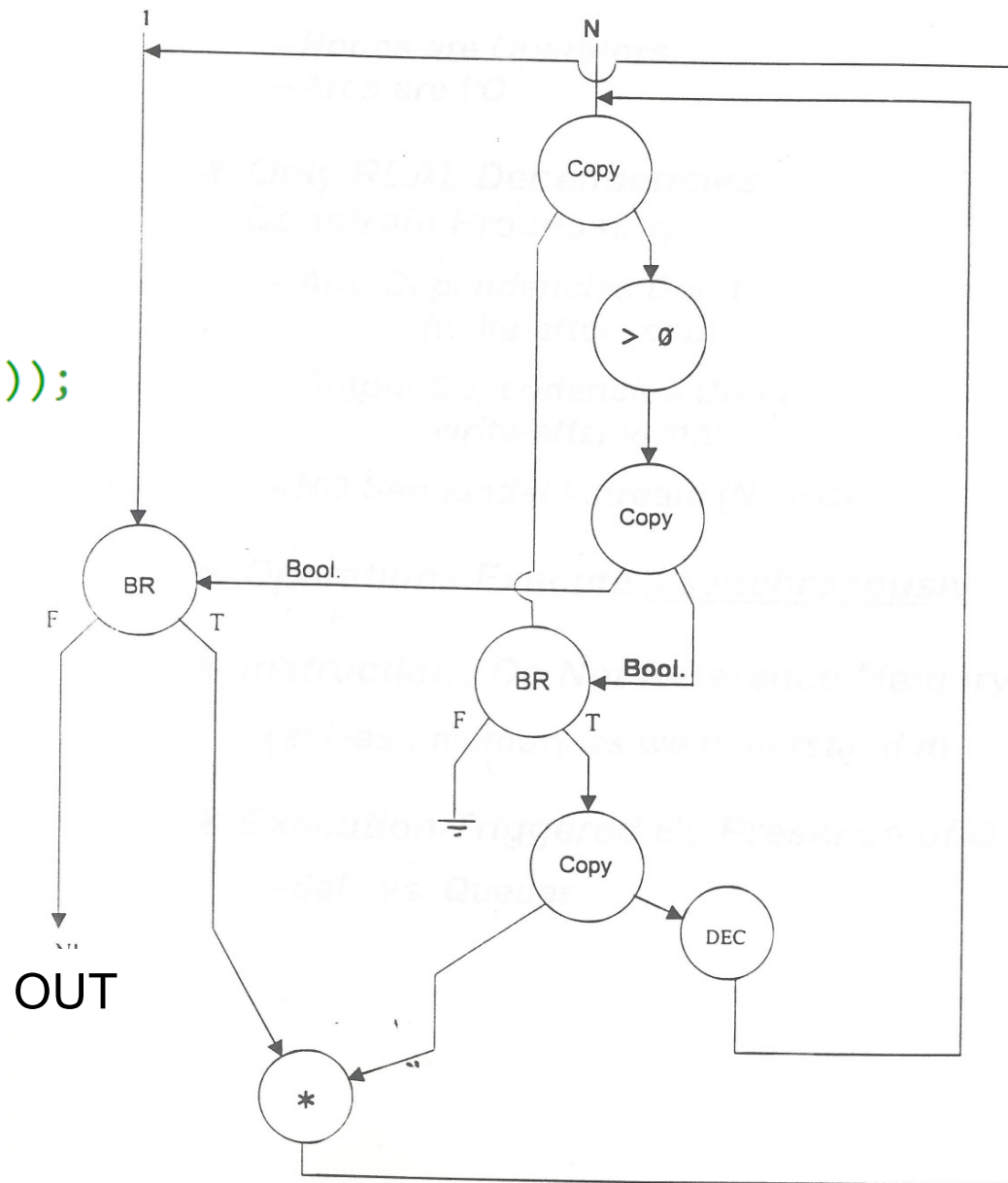


*Barrier Synch



An Example Data Flow Program

```
long factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return(n * factorial(n-1));
}
```



ISA-level Tradeoff: Program Counter

- Do we need a program counter in the ISA?
 - Yes: Control-driven, sequential execution
 - An instruction is executed when the PC points to it
 - PC gets incremented automatically (except for control flow instructions)
 - Precise state where programmer knows what exactly has been executed (e.g. debugging)
 - No: Data-driven, parallel execution
 - An instruction is executed when all its operand values are available (data flow)
 - Programmer does not know precisely what has been executed.
- Tradeoffs: Many high-level ones
 - Ease of programming (for average programmers)?
 - Ease of compilation?
 - Performance: Extraction of parallelism?
 - Hardware complexity?

Problem
Algorithm
Program/Language
Runtime System (VM, OS, MM)
ISA (Architecture)
Microarchitecture
Logic
Circuits
Electrons

ISA vs. Microarchitecture Level Tradeoff

- A similar tradeoff (control-driven vs. data-driven execution) can be made at the microarchitecture level
- 4 ways to design a computer (2-ways per level)
- ISA: Specifies how the programmer sees program (instructions to be executed)
 - Execution is as a sequence of instructions that are executed in an order (semantics)
- Microarchitecture: Specifies how the underlying implementation actually executes instructions
 - Can execute instructions in any order as long as it obeys the semantics specified by the ISA when instruction results are visible to software (programmer sees the right order)

Problem
Algorithm
Program/Language
Runtime System (VM, OS, MM)
ISA (Architecture)
Microarchitecture
Logic
Circuits
Electrons

ISA DESIGN

ISA design

- ISA defines **how** such statements are **translated to machine code**
- Think about a **HLL statement** like

$x[i+1] = x[i] * 30;$

- What information is needed?

Instruction Set Architecture

- Questions every ISA designer must answer
 - How will the processor implement this statement?
 - What operations are available?
 - How many operands does each instruction use?
 - How do we reference the operands?
 - Where are $X[i]$ and i located?
 - What types of operands are supported?
 - How big are those operands
 - Instruction format issues
 - How many bits per instruction?
 - What does each bit or set of bits represent?
 - Are all instructions the same length?

ISA design

- The ultimate **goals** of the **ISA designer** are
 - To create an ISA that allows for **fast hardware implementations**
 - To **simplify choices** for the **compiler**
 - To ensure the **longevity of the ISA** by anticipating future technology trends
- Example **ISAs**: X86; PowerPC, SPARC, ARM, MIPS, IA-64
 - May have **multiple hardware implementations** of the **same ISA**
 - Example: i386, i486, Pentium, Pentium Pro, Pentium II, Pentium III, Pentium IV, i3, i5, i7, i9,...

ISA design

- The ultimate goals of the ISA designer are
 - To create an ISA that allows for fast hardware implementations
 - To simplify choices for the compiler
 - To ensure the longevity of the ISA by anticipating future technology trends
- Example ISAs: X86; PowerPC, SPARC, ARM, MIPS, IA-64
 - May have multiple hardware implementations of the same ISA
 - Example: i386, i486, Pentium, Pentium Pro, Pentium II, Pentium III, Pentium IV, i3, i5, i7, i9,...

Design Goal: Fast Hardware

- From ISA perspective, must **understand how processor executes instruction**
 1. **Fetch** the instruction from memory
 2. **Decode** the instruction
 3. Determine **addresses** for **operands**
 4. **Fetch operands**
 5. **Execute** instruction
 6. **Store** result (and go back to step 1 ...)
- Steps 1, 2, and 5 involve **operation issues**
 - What types of operations are supported?
- Steps 2-6 involve **operand issues**
 - Operand size, number, location
- Steps 1-3 involve **instruction format issues**
 - How many bits in instruction, what does each field mean?

Designing Fast Hardware

- Fast instruction **fetch** and **decode**
 - An ISA perspective
- Fast **operand** access
 - ISA classes: **where** do we **store operands**?
 - **Addressing modes**: how do we specify operand locations?
 - We know **registers** can be used for **fast accesses**
- Fast **execution** of simple operations
 - **Optimize common case**
 - Implementing **single-cycle operations**
 - Dealing with **multi-cycle** operations (has **challenges**)

ELEMENTS OF AN ISA

ISA

- Instructions

- Opcodes, Addressing Modes, Data Types
- Instruction Types and Formats
- Registers, Condition Codes

- Memory

- Address space, Addressability, Alignment
- Virtual memory management

- Call, Interrupt/Exception Handling

- Access Control, Priority/Privilege

- I/O: memory-mapped vs. instr.

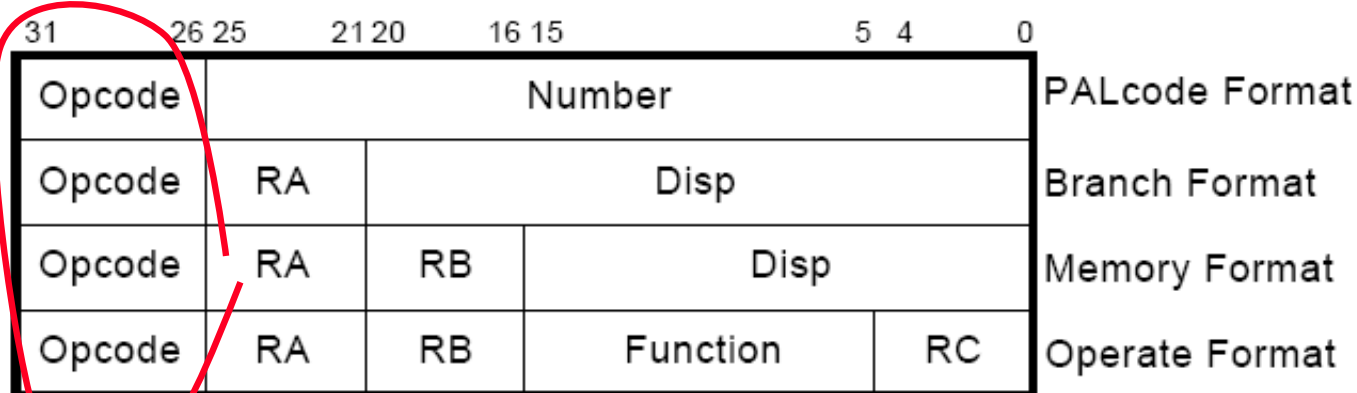
- Task/thread Management

- Power and Thermal Management

- Multi-threading support, Multiprocessor support

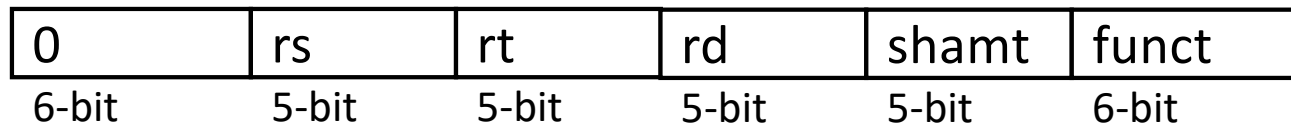
Elements of an ISA: Instruction

- Basic element of the HW/SW interface
- Consists of
 - **opcode**: what the instruction does
 - **operands**: who the instruction does it to
 - Example from **Alpha ISA**:

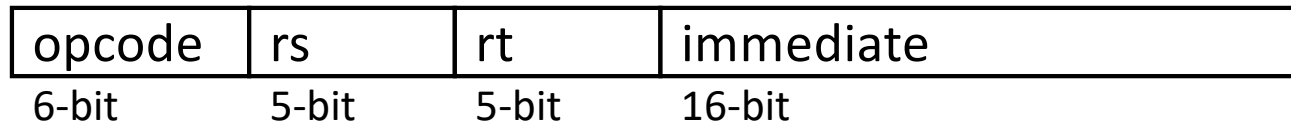


Uniform Decode: same bits are used to represent the operation across different instruction format

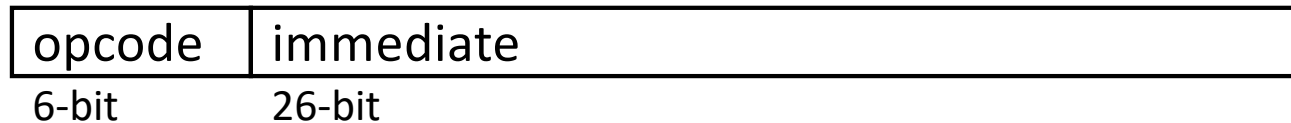
MIPS Instruction Format



R-type



I-type



J-type

ARM Instruction format

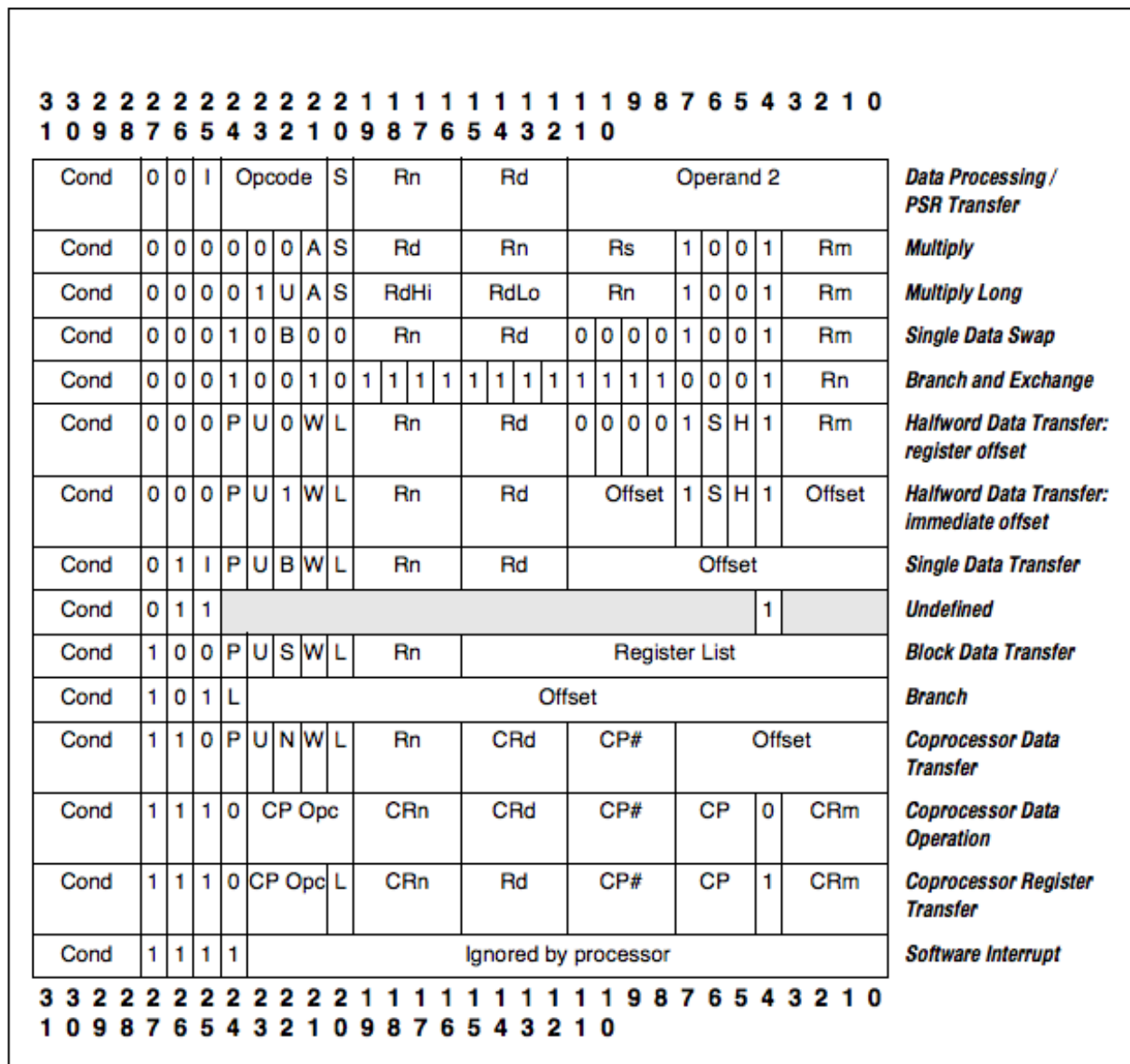


Figure 4-1: ARM instruction set formats

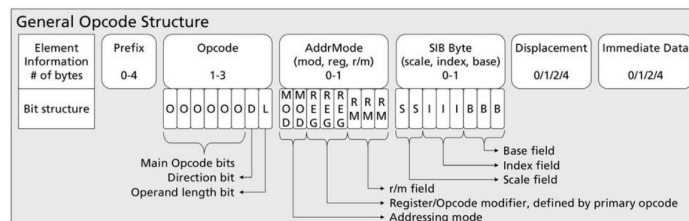
X86 instruction Set Reference

x86 Opcode Structure and Instruction Overview

2 nd 1 st	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	ADD					ES PUSH SS	ES POP SS	OR					CS PUSH DS	TWO BYTE POP DS		
1	ADC					ES SEGMENT OVERRIDE SS	DAA	SBB					CS SEGMENT OVERRIDE DS	DAS		
2	AND					SS	AAA	SUB					DS	AAS		
3	XOR					CMP										
4	INC					DEC										
5	PUSH					POP										
6	PUSHAD	POPAD	BOUND	ARPL	FS	GS	OPERAND SIZE OVERRIDE	ADDRESS SIZE OVERRIDE	PUSH	IMUL	PUSH	IMUL	INS	OUTS		
7	JO	JNO	JB	JNB	JE	JNE	JBE	JA	JS	JNS	JPE	JPO	JL	JGE	JLE	JG
8	ADD/ADC/AND/XOR OR/SBB/SUB/CMP				TEST		XCHG		MOV REG			MOV SREG	LEA	MOV SREG	POP	
9	NOP	XCHG EAX					CWD	CDQ	CALL	WAIT	PUSHFD	POPF	SAHF	LAHF		
A	MOV EAX		MOVS		CMPS		TEST		STOS		LODS		SCAS			
B	MOV															
C	SHIFT IMM		RETN		LES	LDS	MOV IMM		ENTER	LEAVE	RETF		INT3	INT IMM	INTO	IRETD
D	SHIFT 1		SHIFT CL		AAM	AAD	SALC	XLAT	FPU							
E	LOOPNZ		LOOPZ	LOOP	JECXZ		IN IMM		OUT IMM		CALL	JMP	JMPF	JMP SHORT	IN DX	OUT DX
F	LOCK EXCLUSIVE ACCESS	ICE BP	REPNE	REPE	HLT	CMC	TEST/NOT/NEG [i]MUL/[i]DIV		CLC	STC	CLI	STI	CLD	STD	INC DEC	INC/DEC CALL/JMP PUSH

2 nd 1 st	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	(L,S)LDT (L,S)TR VER(R,W)	(L,S)GDT (L,S)IDT (L,S)MSW	LAR	LSL			CLTS		INVD	WBINVD		UD2		NOP		
1	SSE{1,2,3}								Prefetch SSE1	HINT_NOP						
2	MOV CR/DR								SSE{1,2}							
3	WRMSR	RDTSC	RDMRSM	RDPDPC	SYSENTER	SYSEXIT		GETSEC SMX	MOVBE / THREE BYTE		THREE BYTE SSE4					
4	CMOV															
5	SSE{1,2}															
6	MMX, SSE2															
7	MMX, SSE{1,2,3}, VMX												MMX, SSE{2,3}			
8	JO	JNO	JB	JNB	JE	JNE	JBE	JA	JS	JNS	JPE	JPO	JL	JGE	JLE	JG
	Jcc SHORT															
9	SETO	SETNO	SETB	SETNB	SETE	SETNE	SETBE	SETA	SETS	SETNS	SETPE	SETPO	SETL	SETGE	SETLE	SETG
	SETcc															
A	PUSH FS	POP FS	CPUID	BT	SHLD				PUSH GS	POP GS	RSM	BTS	SHRD		*FENCE	IMUL
B	CMPXCHG	LSS	BTR	LFS	LGS	MOVZX		POPCNT	UD	BT BTS BTR BTC		BTC	BSF	BSR	MOVSB	
C	XADD		SSE{1,2}				CMPXCHG	BSWAP								
D	MMX, SSE{1,2,3}															
E	MMX, SSE{1,2}															
F	MMX, SSE{1,2,3}															

Arithmetic & Logic	Prefix
Memory	System & I/O
Stack	No Operation (NOP) / Multiple Instructions / Extended Instruction Set
Control Flow & Conditional	



Addressing Modes

mod	00	01	10	11
r/m	16bit	32bit	16bit	32bit
000	[BX+SI]	[EAX]	[BX+SI+disp8]	[EAX+disp8]
001	[BX+DI]	[ECX]	[BX+DI+disp8]	[ECX+disp8]
010	[BP+SI]	[EDX]	[BP+SI+disp8]	[EDX+disp8]
011	[BP+DI]	[EBX]	[BP+DI+disp8]	[EBX+disp8]
100	[SI]	[SIB]	[SI+disp8]	[SIB+disp8]
101	[DI]	[DIP]	[DI+disp8]	[DIP+disp8]
110	[SI]	[BP]	[SI+disp8]	[BP+disp8]
111	[BX]	[EDX]	[BX+disp8]	[EDX+disp8]

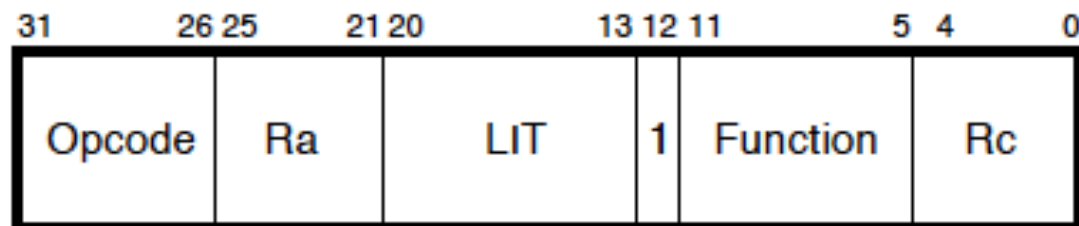
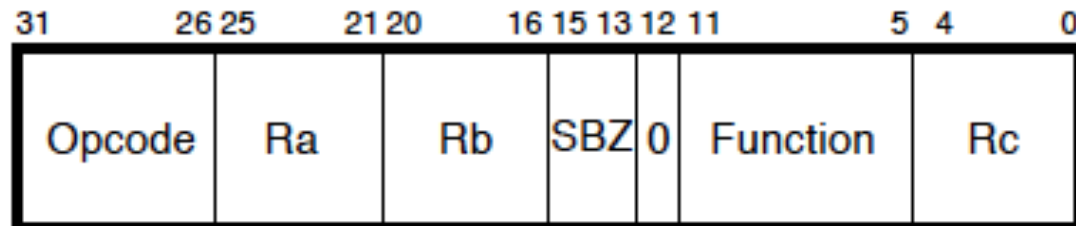
SIB Byte Structure

encoding	scale (2bit)	Index (3bit)	Base (3bit)
000	2 ⁰ =1	[EAX]	EAX
001	2 ¹ =2	[ECX]	ECX
010	2 ² =4	[EDX]	EDX
011	2 ³ =8	[EBX]	EBX
100	--	none	ESP
101	--	[EBP]	disp32 / disp8 [EBP] / disp32 + [EBP]
110	--	[ESI]	ESI
111	--	[EDI]	EDI

SIB value = index * scale + base

Bit Steering (Example: in Alpha)

Figure 3–4: Operate Instruction Format



Bit Steering:

A bit in the instruction **determines** the **interpretation** of other bits

If bit <12> of the instruction is 0, the Rb field specifies a source register operand.

If bit <12> of the instruction is 1, an 8-bit zero-extended literal constant is formed by bits <20:13> of the instruction. The literal is interpreted as a positive integer between 0 and 255 and is zero-extended to 64 bits.

What Must an Instruction Specify?

- Which **operation to perform**: `add r0, r1, r3`
 - Op code: **add, load, branch**, etc.
- **Where** to find the operand or **operands** `add r0, r1, r3`
 - In CPU **registers, memory** cells, **I/O** locations, or part of instruction
- Place to **store result**
 - Again CPU **register** or **memory** cell
- Location of **next instruction**
`add r0, r1, r3`
`br endloop`
 - The default is usually memory cell pointed to by program counter—**PC**: the **next instruction in sequence**
 - **Sometimes** there is **no operand**, or **no result**, or **no next instruction**.
Can you think of **examples**?



Data Flow

Elements of an ISA: Instruction

- Instruction sequencing model
 - Control flow vs. data flow
 - Tradeoffs!
- Instruction processing style
 - Specifies the **number of “operands”** an instruction “operates” on and how it does it
 - 0, 1, 2, 3 address machines
 - 0-address: **stack** machine (**push A, pop A, op**)
 - 1-address: **accumulator** machine (**ld A, st A, op A**)
 - 2-address: 2-operand machine (**one** is **both source and dest**)
 - 3-address: 3-operand machine (source and dest are **separate**)

3, 2, 1, & 0 Address Instructions

- The **classification** is based on **arithmetic instructions** that have **two** operands and **one** result
- The key issue is “**how many** of these **are specified in the instruction**, as opposed to being **implicitly specified**”

3, 2, 1, & 0 Address Instructions (cont.)

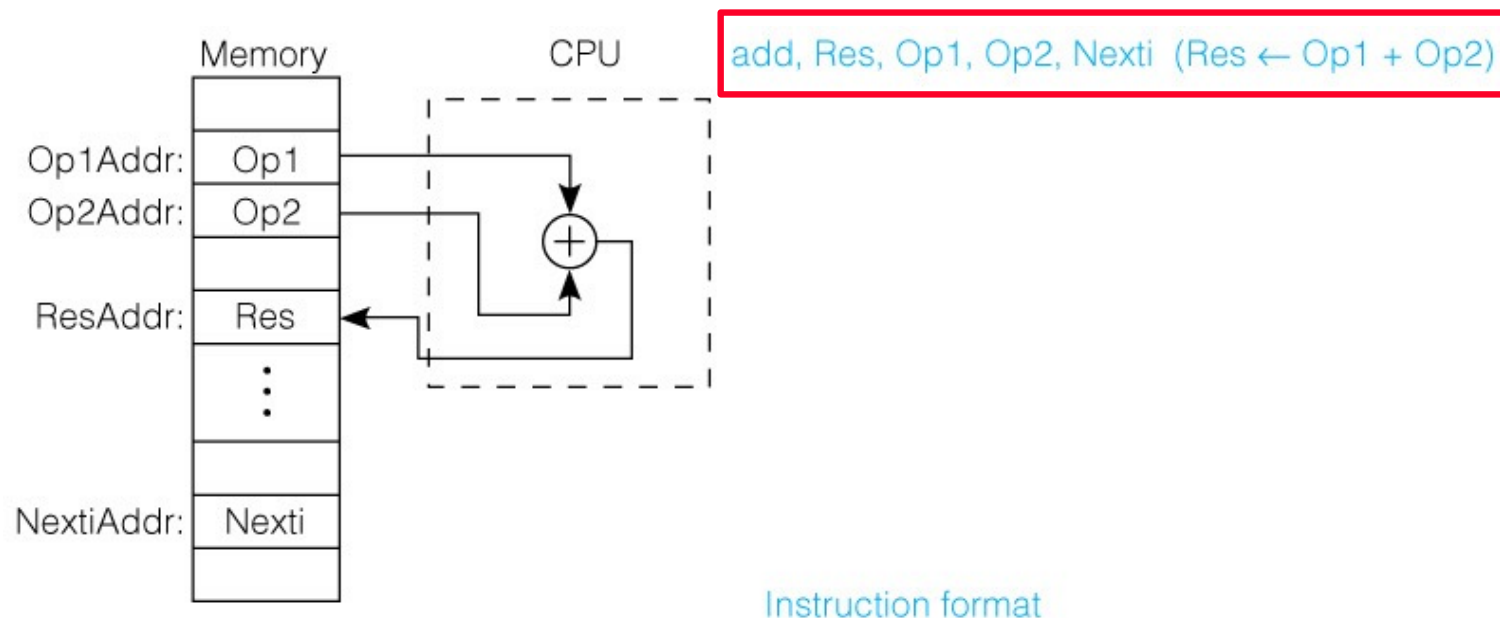
- A 3 address instruction specifies addresses for both operands and the result: **$R \leftarrow Op1 \text{ op } Op2$**
- A 2 address instruction overwrites one operand with the result: **$Op2 \leftarrow Op1 \text{ op } Op2$**
- A 1 address instruction has a register, called the accumulator register to hold one operand & the result (no address needed):
 $Acc \leftarrow Acc \text{ op } Op1$
- A 0 address uses a CPU register stack to hold both operands and the result: **$TOS \leftarrow TOS \text{ op } SOS$**
where TOS is Top Of Stack, SOS is Second On Stack)

Size of Instructions in Bytes

- Example assumes **16 MB** (24-bit addresses) and **200 instructions** (8 bis)

The 4 Address Instruction

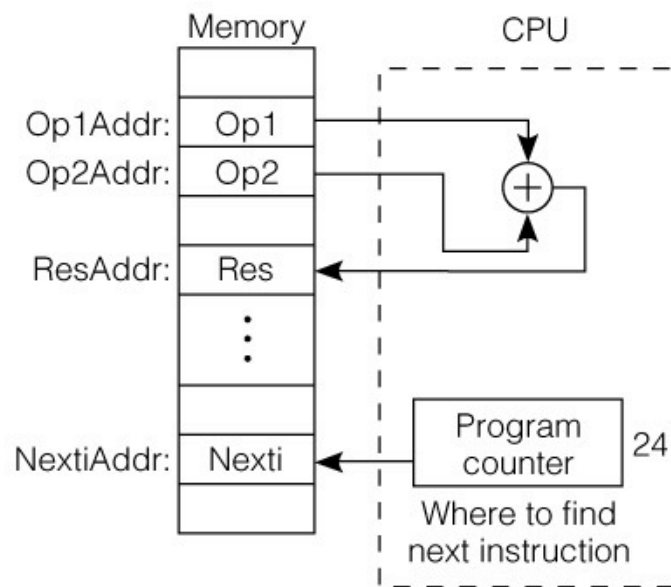
- Explicit addresses for operands, result & next instruction



Bits:	8	24	24	24	24
	add	ResAddr	Op1Addr	Op2Addr	NextiAddr
	Which operation	Where to put result	Where to find operands		Where to find next instruction

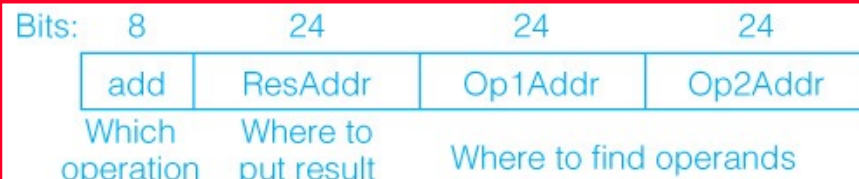
The 3 Address Instruction

- Address of **next instruction kept in** a processor state register—the **PC** (Except for explicit Branches/Jumps)
- Rest of addresses in instruction** (savings in instruction word size)



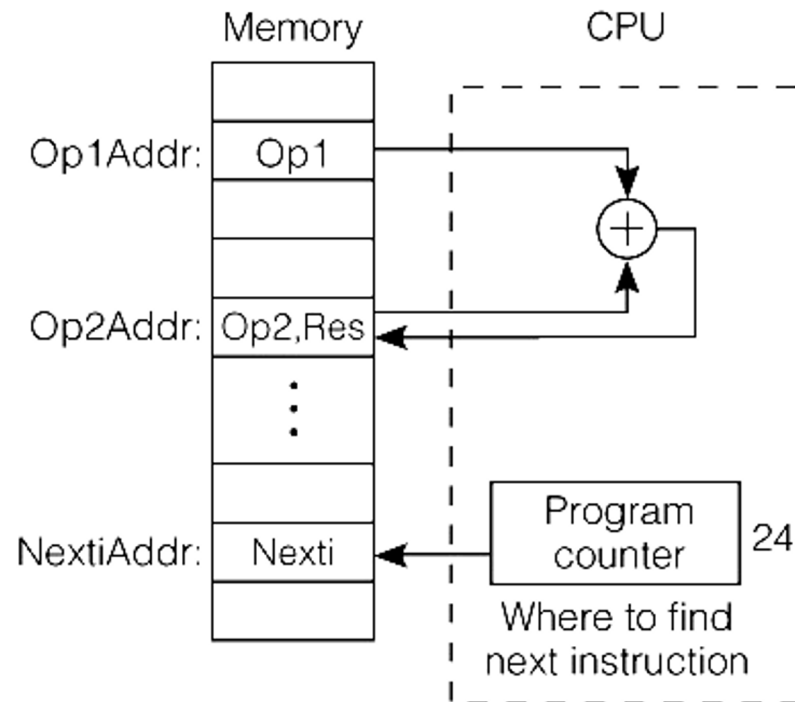
`add, Res, Op1, Op2 (Res ← Op2 + Op1)`

Instruction format

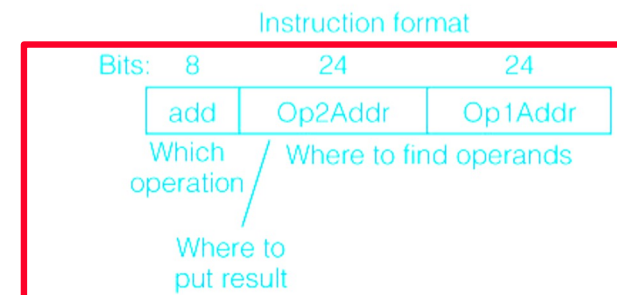


The 2 Address Instruction

- Be aware of the difference between address (Op1Addr) and the data stored at that address, Op1.
- **Result overwrites Operand 2, Op2, with result, Res**
- This format needs only 2 addresses in the instruction but there is **less choice in placing data**

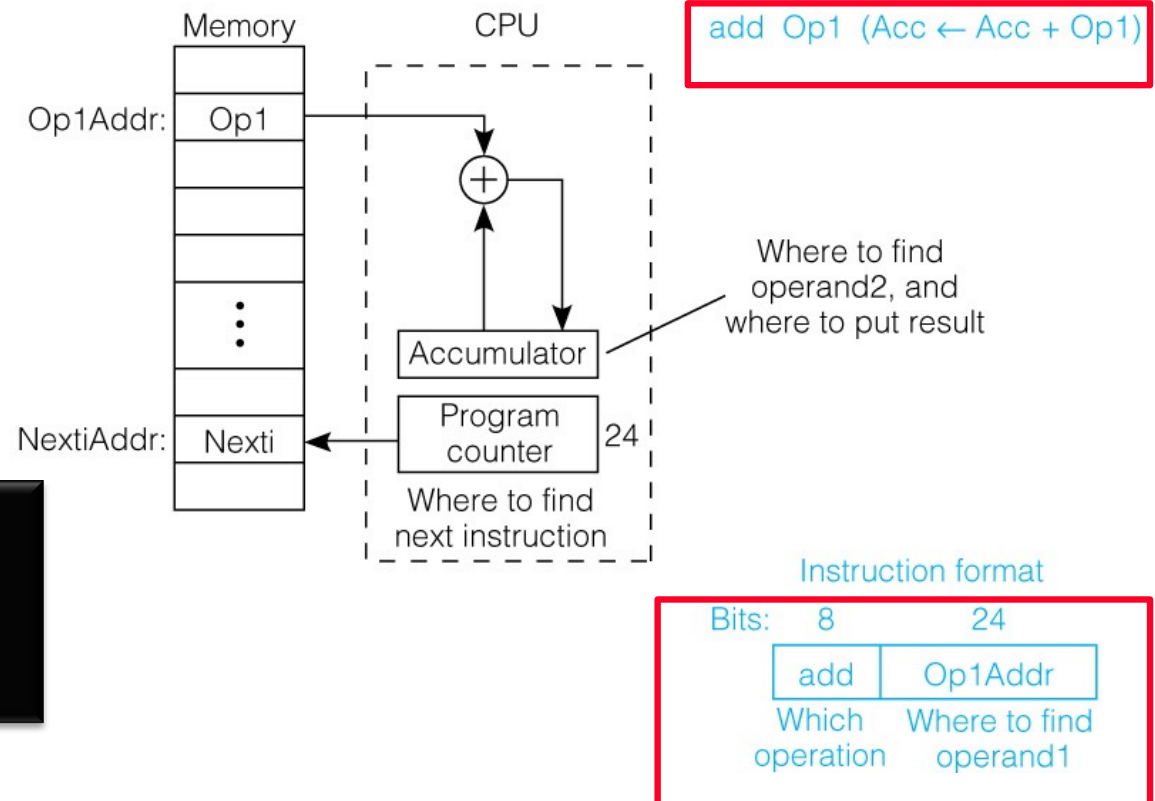


add Op2, Op1 ($\text{Op2} \leftarrow \text{Op2} + \text{Op1}$)



1 Address Instructions

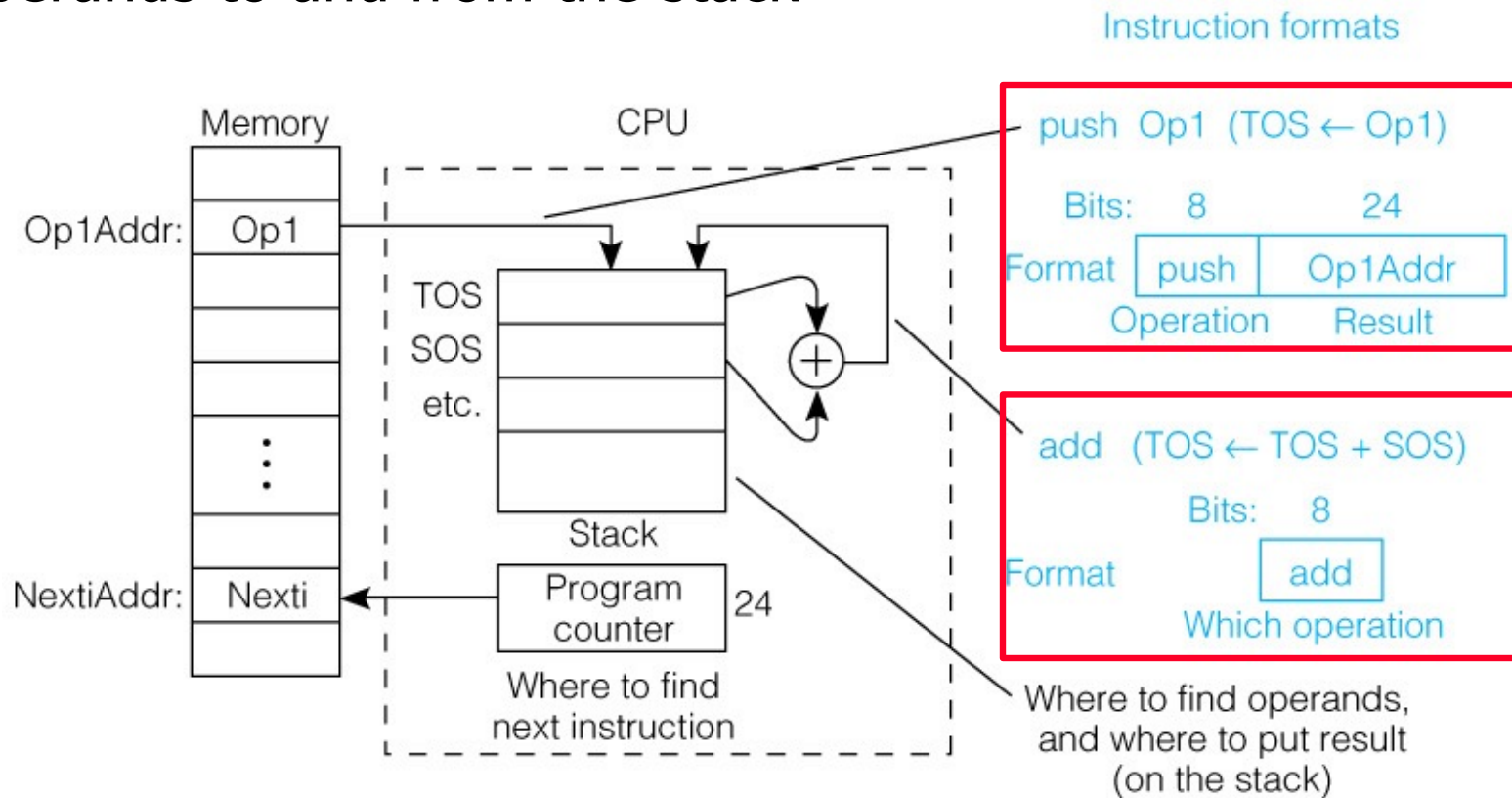
- Special CPU register, the **accumulator**, supplies **one operand** and **stores result**
- One **memory address** used for other **operand**



We now need **instructions** to **load and store** operands:
LDA OpAddr
STA OpAddr

The 0 Address Instruction

- Uses a **push down stack** in CPU
- Arithmetic uses stack for **both operands**. The **result** replaces them on the **TOS**
- Computer must have a **1 address instruction** to **push** and **pop** operands to and from the stack



Example 1: Expression evaluation for 3-0 address instructions

- Evaluate $a = (b+c)*d-e$ for 3- 2- 1- and 0-address machines

3-Address	2-Address	Accumulator	Stack

Example 1: Expression evaluation for 3-0 address instructions

- Evaluate $a = (b+c)*d-e$ for 3- 2- 1- and 0-address machines

3-Address	2-Address	Accumulator	Stack
add a,b,c mpy a,a,d sub a,a,e	load a,b add a,c mpy a,d sub a,e	lda b add c mpy d sub e sta a	push b push c add push d mpy push e sub pop a

ISA

- Instructions
 - Registers, Condition Codes
 - Instruction Types and Formats
 - Opcodes, Addressing Modes, Data Types
- Memory
 - Address space, Addressability, Alignment
 - Virtual memory management
- Call, Interrupt/Exception Handling
- Access Control, Priority/Privilege
- I/O: memory-mapped vs. instr.
- Task/thread Management
- Power and Thermal Management
- Multi-threading support, Multiprocessor support

Elements of an ISA: Registers

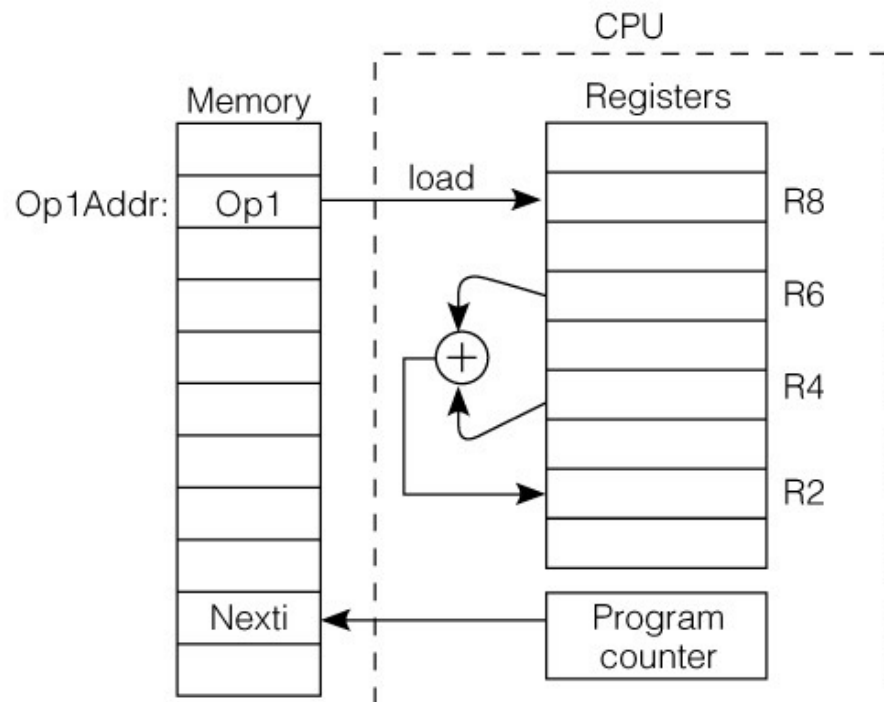
- How many registers? How long each register?
- Why is having registers a good idea?
 - Because programs exhibit a characteristic called **data locality**
 - A **compiler** can **analyze** the **code** and decide to place some values in a **fast** (and close to CPU) **memory**
 - A **recently** produced/accessed **value** is likely to be **used more** than once (**temporal locality**)
 - Storing that value **in a register** eliminates the need to **go to memory** each time that value is needed

Principle of locality

- Programs **don't access data randomly**—they display **locality** in two forms
 - **Temporal** locality: if you access a memory location (e.g., 1000), you are **more likely** to **re-access that location again** than some random location
 - And so **local fast memories** on the CPU were born (**registers** were born)
 - **Spatial** locality: if you access a memory location (e.g., 1000), you are **more likely** to **access a location near it** (e.g., 1001) than some random location
 - And so **local fast memories** close to the CPU were born (**Cache Memory**)

General Register Machines

- It is the **most common choice** in today's general purpose computers
- **Which register** is specified by small "**address**" (3 to 6 bits for 8 to 64 registers)
- Load and store have **one long & one short address: 1 1/2 addresses**
- 2-Operand arithmetic instruction has **3 "half" addresses**



Instruction formats

load R8, Op1 ($R8 \leftarrow Op1$)

load	R8	Op1Addr
------	----	---------

add R2, R4, R6 ($R2 \leftarrow R4 + R6$)

add	R2	R4	R6
-----	----	----	----

Real Machines are Not So Simple

- Most real machines have a mixture of 3, 2, 1, 0, 1 **1/2 address** instructions
- A distinction can be made on whether arithmetic instructions use data from memory
- If **ALU instructions** only use **registers** for **operands** and **result**, machine type is **load-store**
 - Only load and store instructions reference memory
- Other machines have a mix of **register-memory** and **memory-memory** instructions

More ISA Examples

- PDP-11: A 2-address machine
 - PDP-11 ADD: 4-bit opcode, 2 6-bit operand specifiers
 - X86: A 2-address (memory/memory) machine
 - Alpha: A 3-address (load/store) machine
 - MIPS
 - ARM
- } A 3-address

Instructions Classes

- Data movement instructions
 - Move data from a memory location or register to another memory location or register without changing its form
 - Load—source is memory and destination is register
 - Store—source is register and destination is memory
- Arithmetic and logic (ALU) instructions
 - Changes the form of one or more operands to produce a result stored in another location
 - Add, Sub, Shift, etc.
- Branch instructions (control flow instructions)
 - Any instruction that alters the normal flow of control from executing the next instruction in sequence
 - Br Loc, Brz Loc2,—unconditional or conditional branches⁵⁶

Data Movement Instructions

- Lots of **variation**, even with one instruction type
- Notice differences in direction of data flow **left-to-right** or **right-to-left**

Instruct.	Meaning	Machine
$\overrightarrow{\text{MOV } A, B}$	Move 16 bits from mem. loc. A to loc. B	VAX11
$\overleftarrow{\text{lwz } R3, A}$	Move 32 bits from mem. loc. A to reg. R3	PPC601
$\overleftarrow{\text{li } \$3, 455}$	Load the 32 bit integer 455 into reg. 3	MIPS R3000
$\overrightarrow{\text{mov } R4, \text{dout}}$	Move 16 bits from R4 to out port dout	DEC PDP11
$\overleftarrow{\text{IN, AL, KBD}}$	Load a byte from in port KBD to accum.	Intel Pentium
$\overrightarrow{\text{LEA. L (A0), A2}}$	Load address pointed to by A0 into A2	M68000

Arithmetic and Logic Instructions

Instruction	Meaning	Machine
MULF A, B, C	multiply the 32-bit floating point values at mem loc'ns. A and B, store at C	VAX11
nabs r3, r1	Store abs value of r1 in r3	PPC601
ori \$2, \$1, 255	Store logical OR of reg \$ 1 with 255 into reg \$2	MIPS R3000
DEC R2	Decrement the 16-bit value stored in reg R2	DEC PDP11
SHL AX, 4	Shift the 16-bit value in reg AX left by 4 bits	Intel 8086

Notice again the complete dissimilarity of both **syntax** and **semantics**

Branch Instructions

Instruction	Meaning	Machine
BLSS A, Tgt	Branch to address Tgt if the least significant bit of mem loc'n. A is set (i.e. = 1)	VAX11
bun r2	Branch to location in R2 if result of previous floating point computation was Not a Number (NaN)	PPC601
beq \$2, \$1, 32	Branch to location (PC + 4 + 32) if contents of \$1 and \$2 are equal	MIPS R3000
SOB R4, Loop	Decrement R4 and branch to Loop if R4 $\neq$ 0	DEC PDP11
JCXZ Addr	Jump to Addr if contents of register CX = 0.	Intel 8086
Eieio	Enforce in-order execution of I/O	PowerPC

Elements of an ISA: Addressing Modes

- An **addressing mode** is hardware support for a useful way of **determining a memory address**
- **Different addressing modes solve different HLL problems**
 - Some addresses may be known at **compile time**, e.g. **global** vars.
 - Others may not be known until **run time**, e.g. **pointers**
 - Addresses may **have to be computed**:
 - Record (**struct**) components: variable base (full address) + const.(small)
 - **Array** components: const. base (full address) + index var.(small)
 - Possible to store **constant values** w/o using another memory cell by storing them **with or adjacent to the instruction** itself

Common Addressing Modes

(a) Immediate addressing:

instruction contains
the operand

Instr

Op	3
----	---

load #3,

Common Addressing Modes

(a) Immediate addressing:

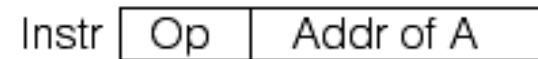
instruction contains
the operand



load #3, ...

(b) Direct addressing:

instruction contains
address of operand



load A, ...

Memory



Common Addressing Modes

(a) Immediate addressing:

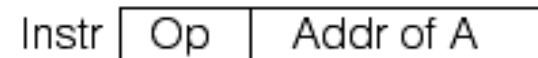
instruction contains
the operand



load #3, ...

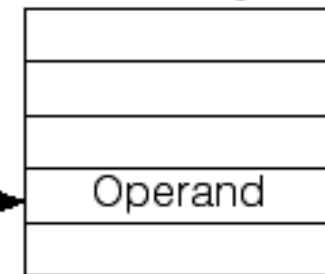
(b) Direct addressing:

instruction contains
address of operand



load A, ...

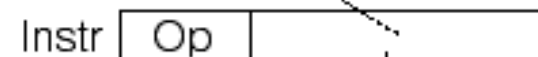
Memory



(c) Indirect addressing:

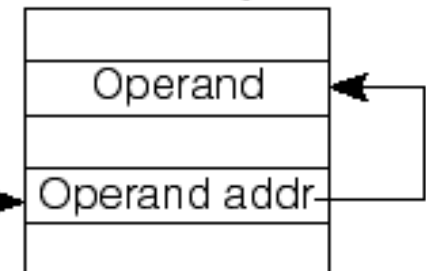
instruction contains
address of address
of operand

Address of address of A



load (A), ...

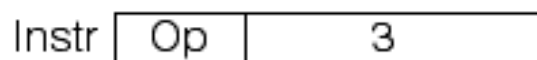
Memory



Common Addressing Modes

(a) Immediate addressing:

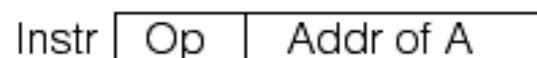
instruction contains
the operand



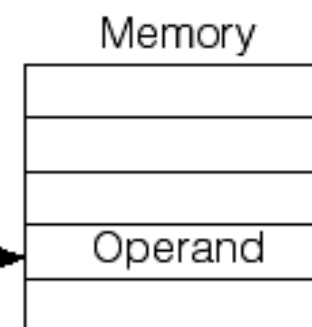
load #3, ...

(b) Direct addressing:

instruction contains
address of operand



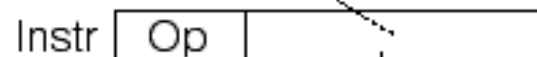
load A, ...



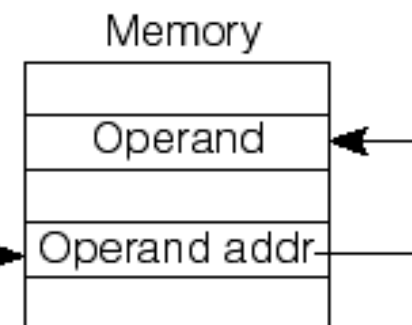
(c) Indirect addressing:

instruction contains
address of address
of operand

Address of address of A



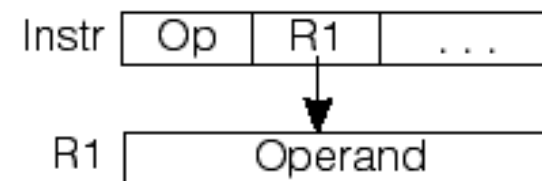
load (A), ...



(d) Register direct addressing:

register contains operand

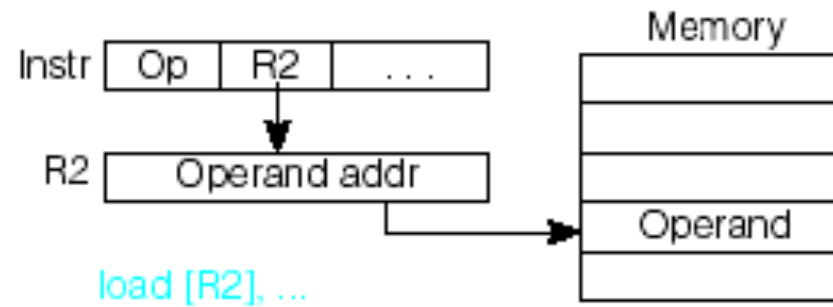
load R1, ...



Common Addressing Modes

(e) Register indirect addressing:

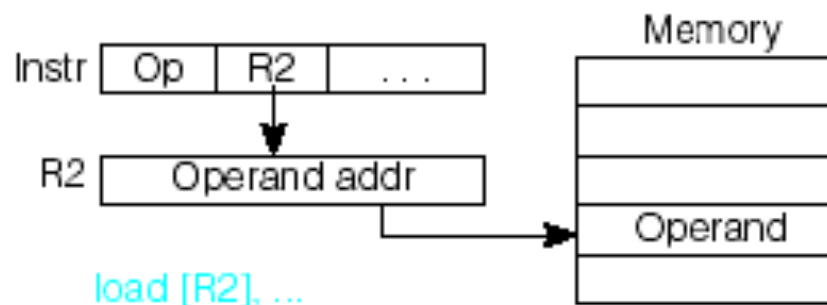
register contains address
of operand



Common Addressing Modes

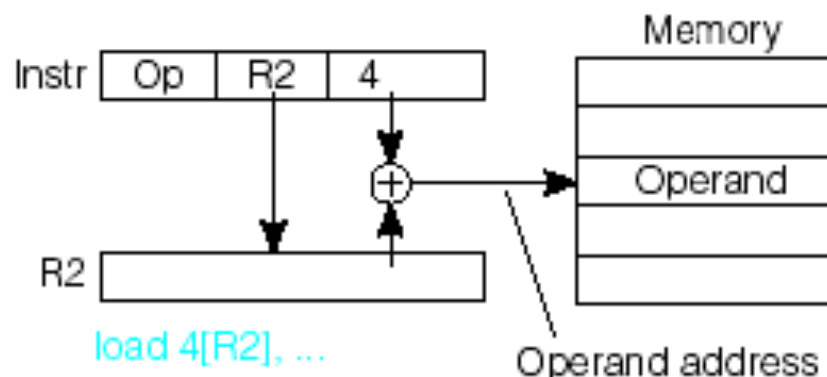
(e) Register indirect addressing:

register contains address
of operand



(f) Displacement (based or indexed) addressing:

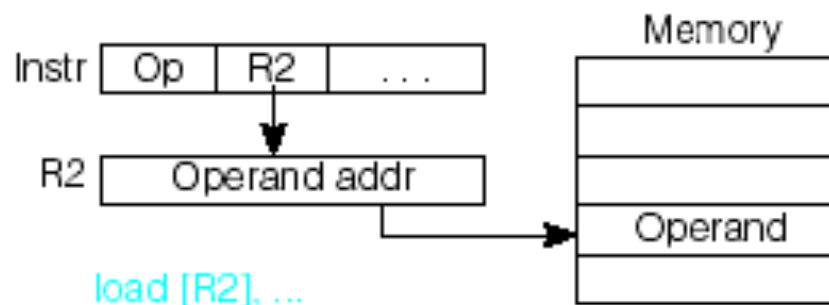
address of operand =
register + constant



Common Addressing Modes

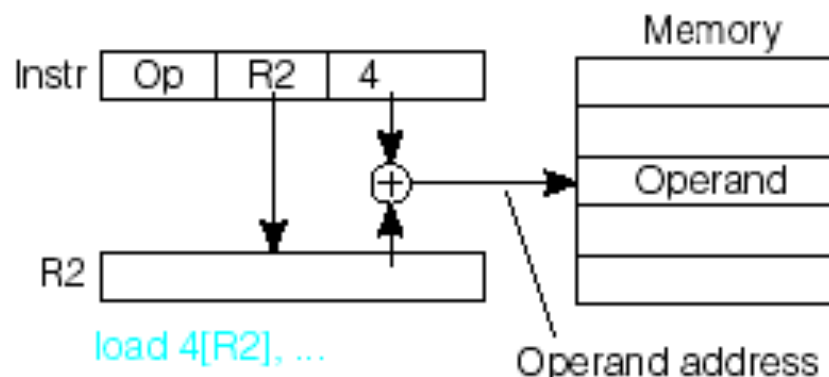
(e) Register indirect addressing:

register contains address
of operand



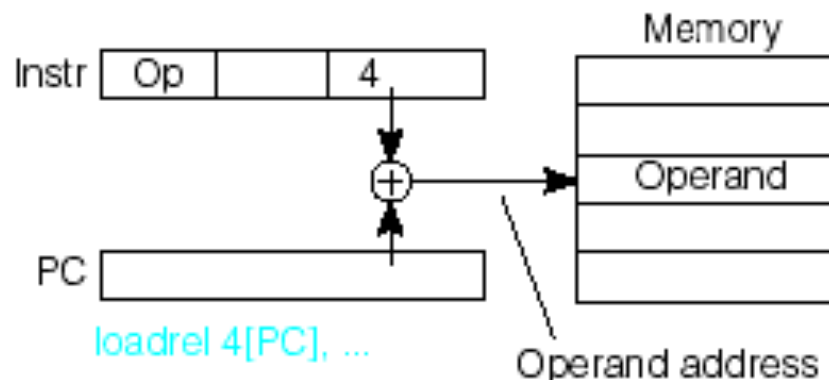
(f) Displacement (based or indexed) addressing:

address of operand =
register + constant



(g) Relative addressing:

address of operand =
PC + constant



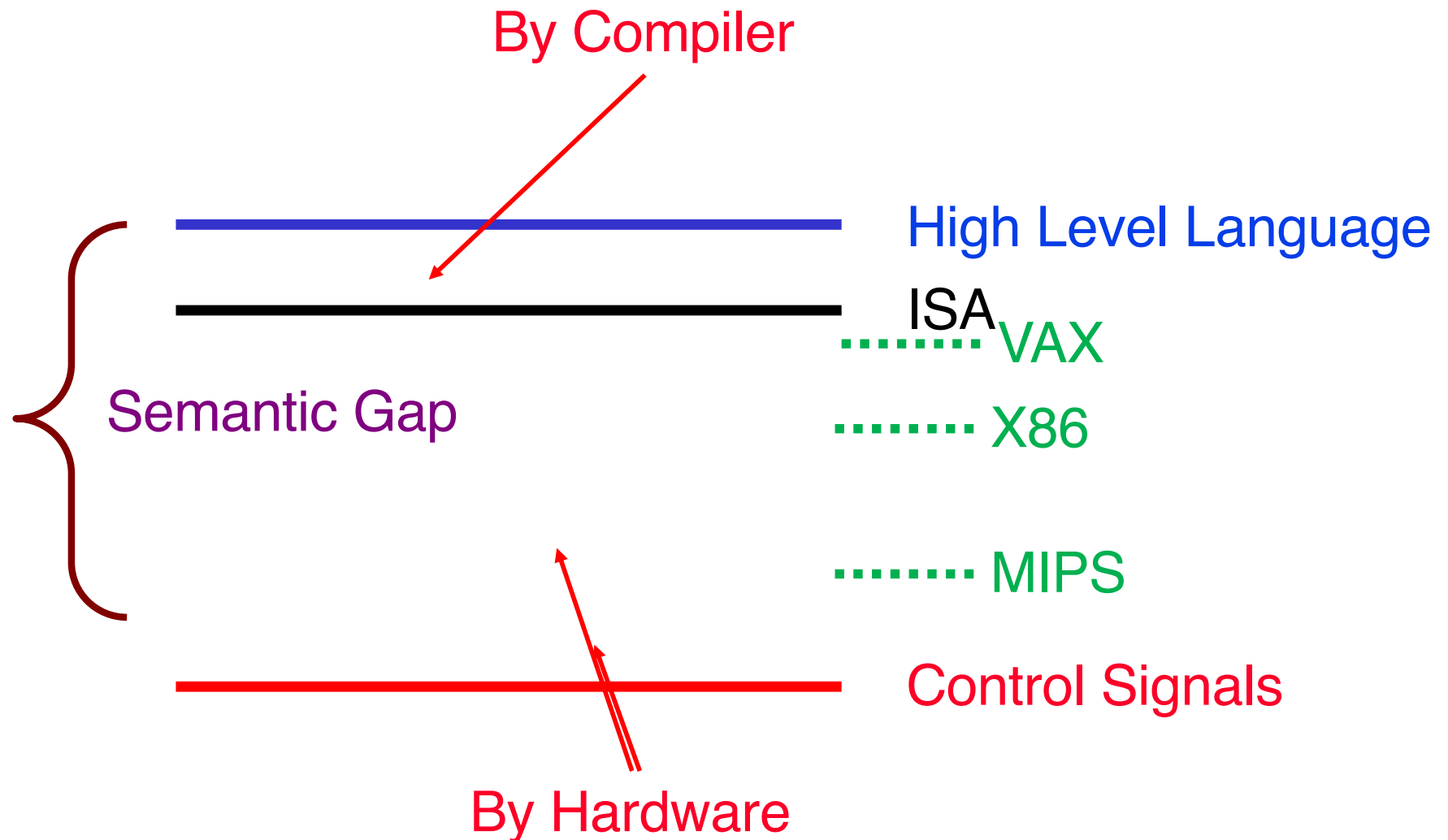
Elements of an ISA: Data Types

- **Data Types:** Representation of information for which there are instructions that operate on the representation
- Integer, floating point, character, binary, decimal, BCD
- Doubly linked list, queue, string, bit vector, stack
 - VAX: INSQUEUE and REMQUEUE instructions on a doubly linked list or queue; FINDFIRST
 - Digital Equipment Corp., "VAX11 780 Architecture Handbook," 1977.
 - X86: SCAN opcode operates on character strings; PUSH/POP

Data Type Tradeoffs

- What is the **benefit** of having **more or high-level (richer) data types** in the ISA?
 - Better program(er) **support**
- What is the **disadvantage**?
 - Extra hardware **cost**
- **Think compiler/programmer vs. microarchitecture**
- Concept of **semantic gap** (support for **rich datatypes**)
 - **Data** types **coupled tightly** to the **semantic** level, or complexity of instructions
- **Example**: Early RISC architectures vs. Intel 432
 - Early RISC: **Only integer** data type (then added **floating point** support **later**)
 - Intel 432: **Object** data type, capability based machine (**very complex datatypes**)

Semantic Gap



ISA-level Tradeoffs: Semantic Gap

- Where to place the ISA? Semantic gap
 - Closer to high-level language (HLL) → Small semantic gap, complex instructions
 - Closer to hardware control signals? → Large semantic gap, simple instructions

How High or Low Can You Go?

- Very large semantic gap
 - Each instruction specifies the complete set of control signals in the machine
 - Compiler generates control signals
- Very small semantic gap
 - ISA is (almost) the same as high-level language
 - Java machines, LISP machines, object-oriented machines

ISA

- Instructions
 - Opcodes, Addressing Modes, Data Types
 - Instruction Types and Formats
 - Registers, Condition Codes
- Memory
 - Address space, Addressability, Alignment
 - Virtual memory management
- Call, Interrupt/Exception Handling
- Access Control, Priority/Privilege
- I/O: memory-mapped vs. instr.
- Task/thread Management
- Power and Thermal Management
- Multi-threading support, Multiprocessor support

Elements of an ISA: Memory Organization

- **Address space**: How many **uniquely identifiable** locations in memory
- **Addressability**: How much data does each uniquely identifiable location store
 - **Byte addressable**: most ISAs, characters are **8 bits**
 - **Bit addressable**: Burroughs 1700. Why? (OS Support)
 - **64-bit addressable**: Some supercomputers. Why? They **operate on values**
 - **32-bit addressable**: First Alpha (**added load/store byte later**)
- Support for **virtual memory**

Endianness

- Sequential **order** in which **bytes are arranged** into larger numerical values, when **stored** in computer memory or secondary storage, **or** when **transmitted** over digital links
- **Numbers** are always **displayed** the **same way** still they are **not stored** in the **same way** in memory:
 - **Big endian (big end first)**: **most significant** byte is stored at the **lowest** byte address (Ex: 68000, PowerPC, Sun SPARC)
 - **Little endian (little end first)**: **least significant** byte is stored at the **lowest** address (Ex: x86, DEC VAX, Alpha)
- **Significance**:
 - If **two processors** with **different conventions** use a local area network, a disk drive, etc., **YOU** need to **pay attention** to endianness

Endianness (cont.)

0	12
1	34
2	56
3	78
4	AB
5	CD
6	EF
7	01

Big endian

BOTH store same data:
word 12345678 is at
location 0,
word ABCDEF01 is at
location 4

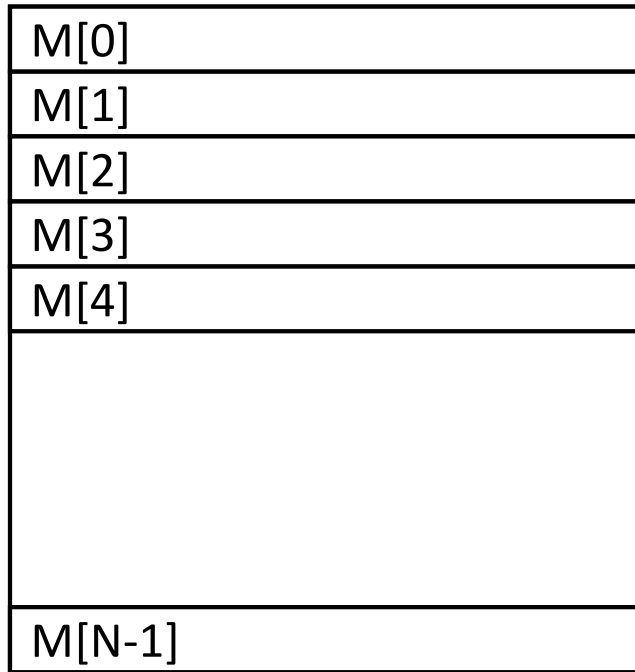
0	78
1	56
2	34
3	12
4	01
5	EF
6	CD
7	AB

Little endian

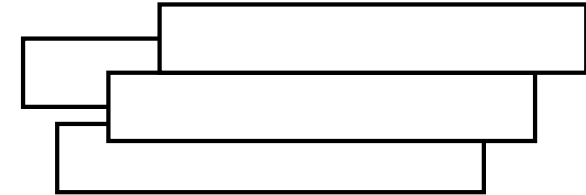
Elements of an ISA: Data Access Support

- Load/store vs. memory/memory architectures
 - Load/store architecture: arithmetic/logic instructions operate only on registers
 - E.g., MIPS, ARM and many RISC ISAs
 - Memory/memory architecture: arithmetic/logic instructions can operate on memory locations as well
 - +Not as many instructions (e.g. copy a big block of memory)
 - - More complex HW (harder for microarchitecture)
 - E.g., x86, VAX and many CISC ISAs

Programmer Visible (Architectural) State



Memory:
array of storage locations
indexed by an address



Registers:

- given special names in the ISA (as opposed to addresses)
- general vs. special purpose

Program Counter:

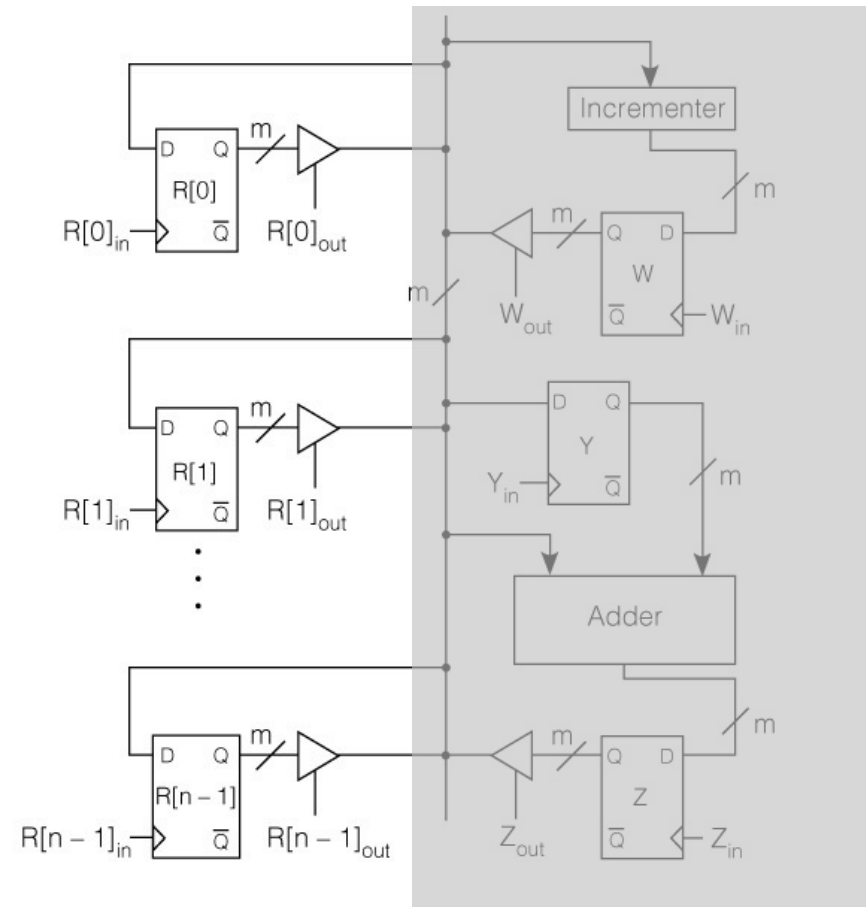
memory address
of the next instruction

Instructions (and programs) specify how to transform the values of programmer visible state

Aside: Programmer Invisible State

- Describes **Microarchitecture** details
- Programmer **cannot access this directly**

- E.g. cache state
- E.g. pipeline registers



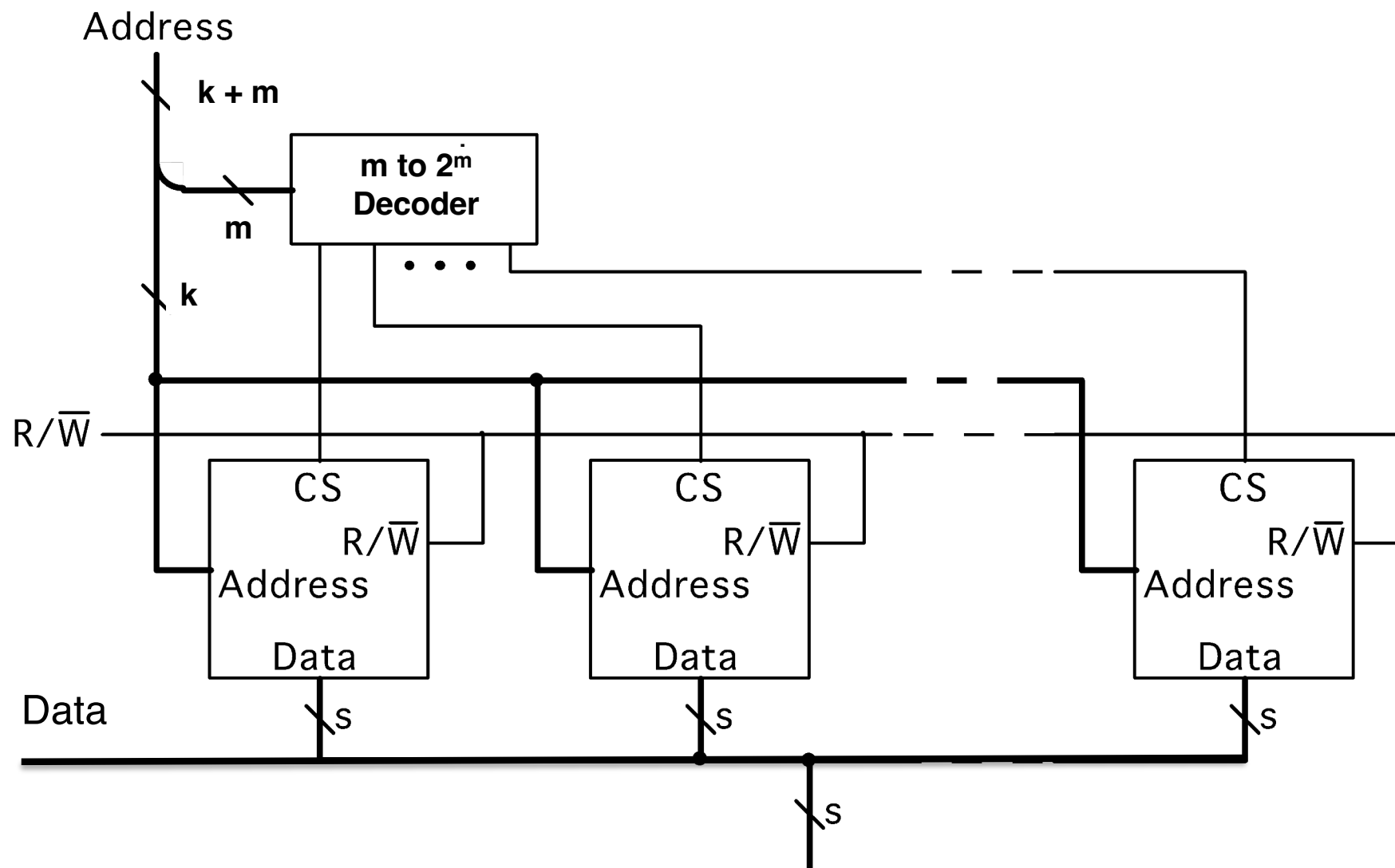
ISA

- Instructions
 - Opcodes, Addressing Modes, Data Types
 - Instruction Types and Formats
 - Registers, Condition Codes
- Memory
 - Address space, Addressability, Alignment
 - Virtual memory management
- Call, Interrupt/Exception Handling
- Access Control, Priority/Privilege
- I/O: memory-mapped vs. instr.
- Task/thread Management
- Power and Thermal Management
- Multi-threading support, Multiprocessor support

Elements of an ISA: I/O Support

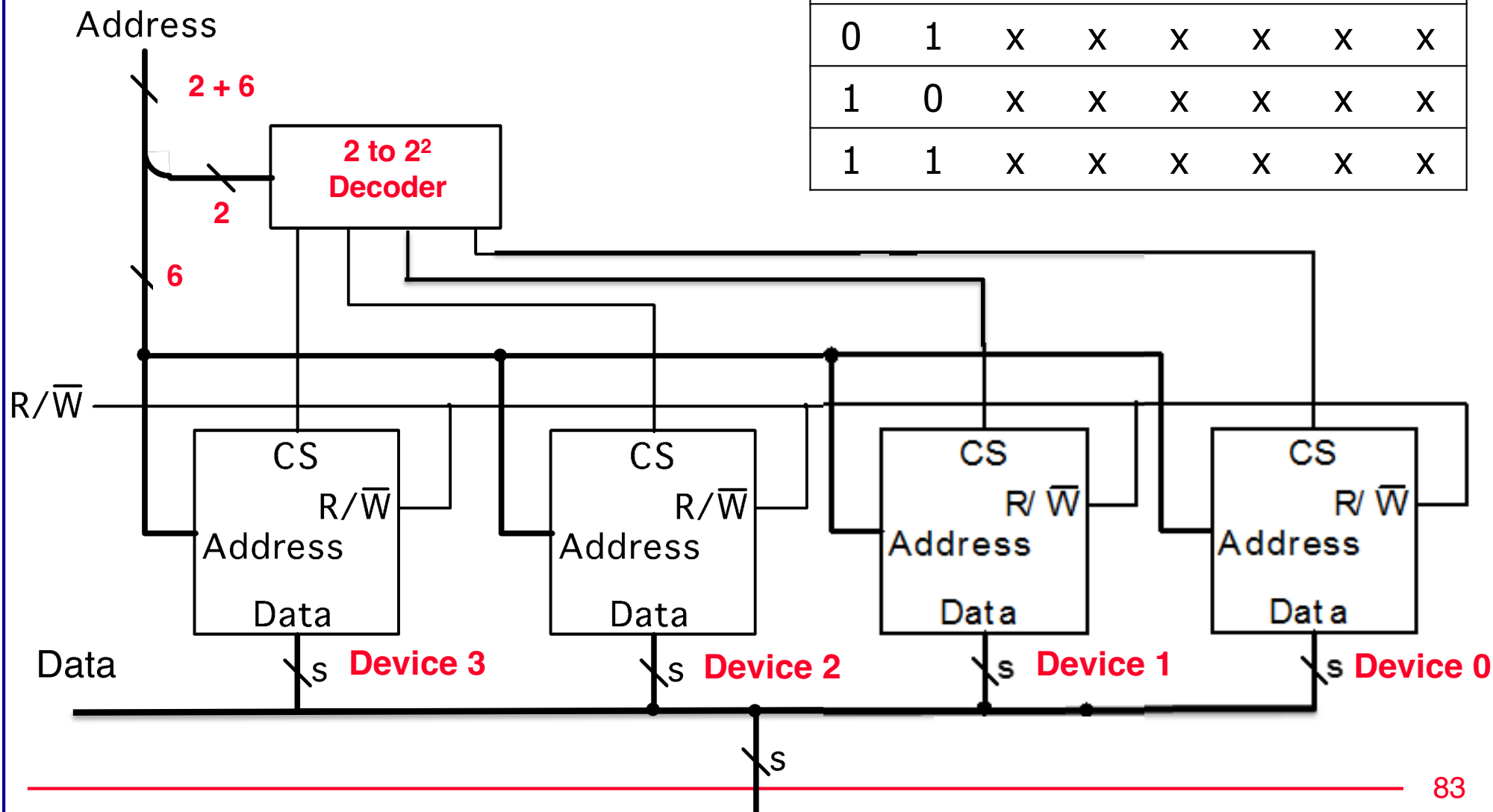
- How to interface with I/O devices
 - Memory mapped I/O
 - A region of **memory** is **mapped** to I/O devices
 - I/O operations are **loads and stores** to those locations
 - Special I/O instructions
 - **IN and OUT instructions** in x86 deal with ports of the chip
 - Tradeoffs?
 - Which one is **more general purpose**? **Memory mapped**

Memory mapped I/O

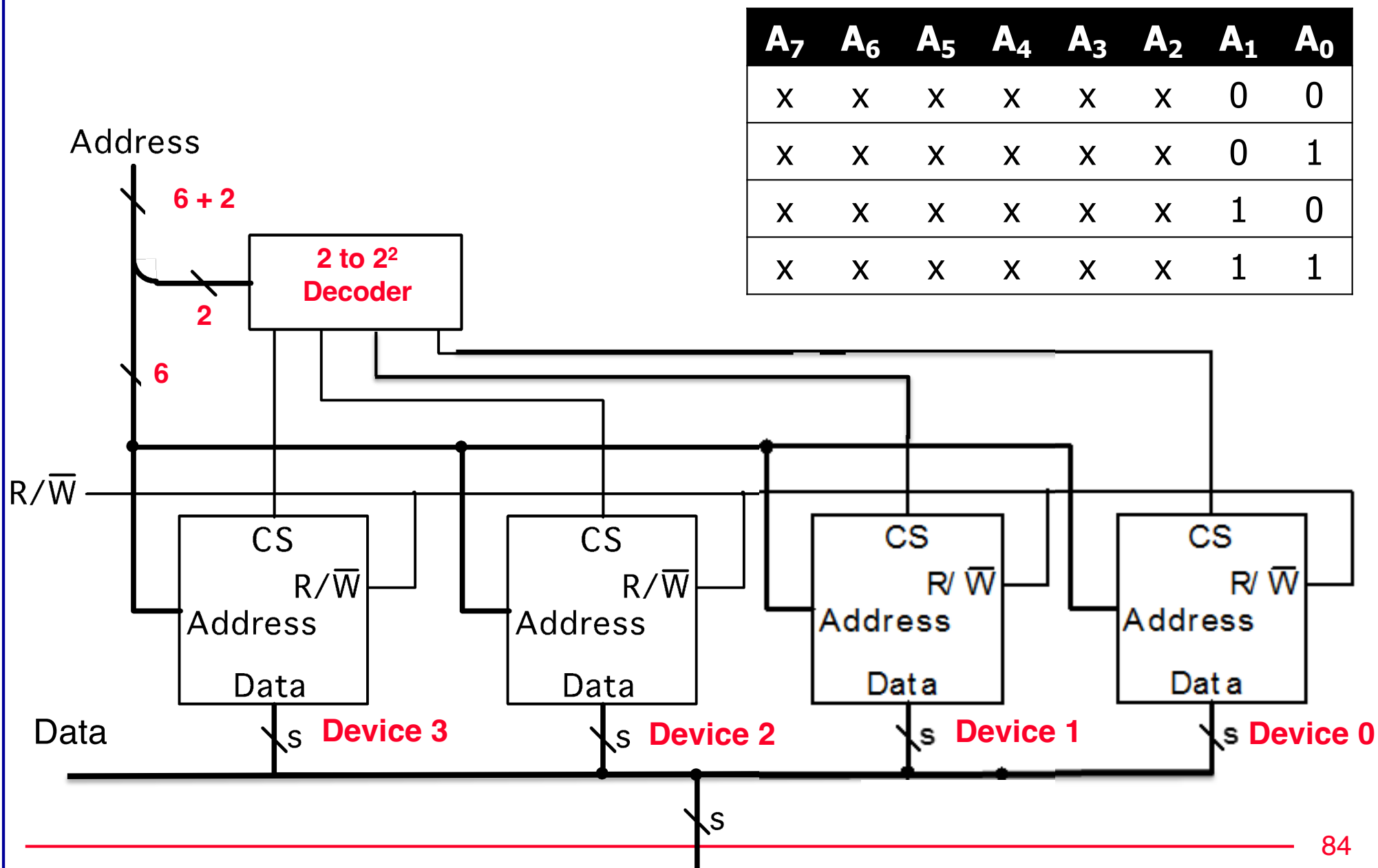


Memory mapped I/O (Example)

A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
0	0	x	x	x	x	X	x
0	1	x	x	x	x	x	x
1	0	x	x	x	x	x	x
1	1	x	x	x	x	x	x



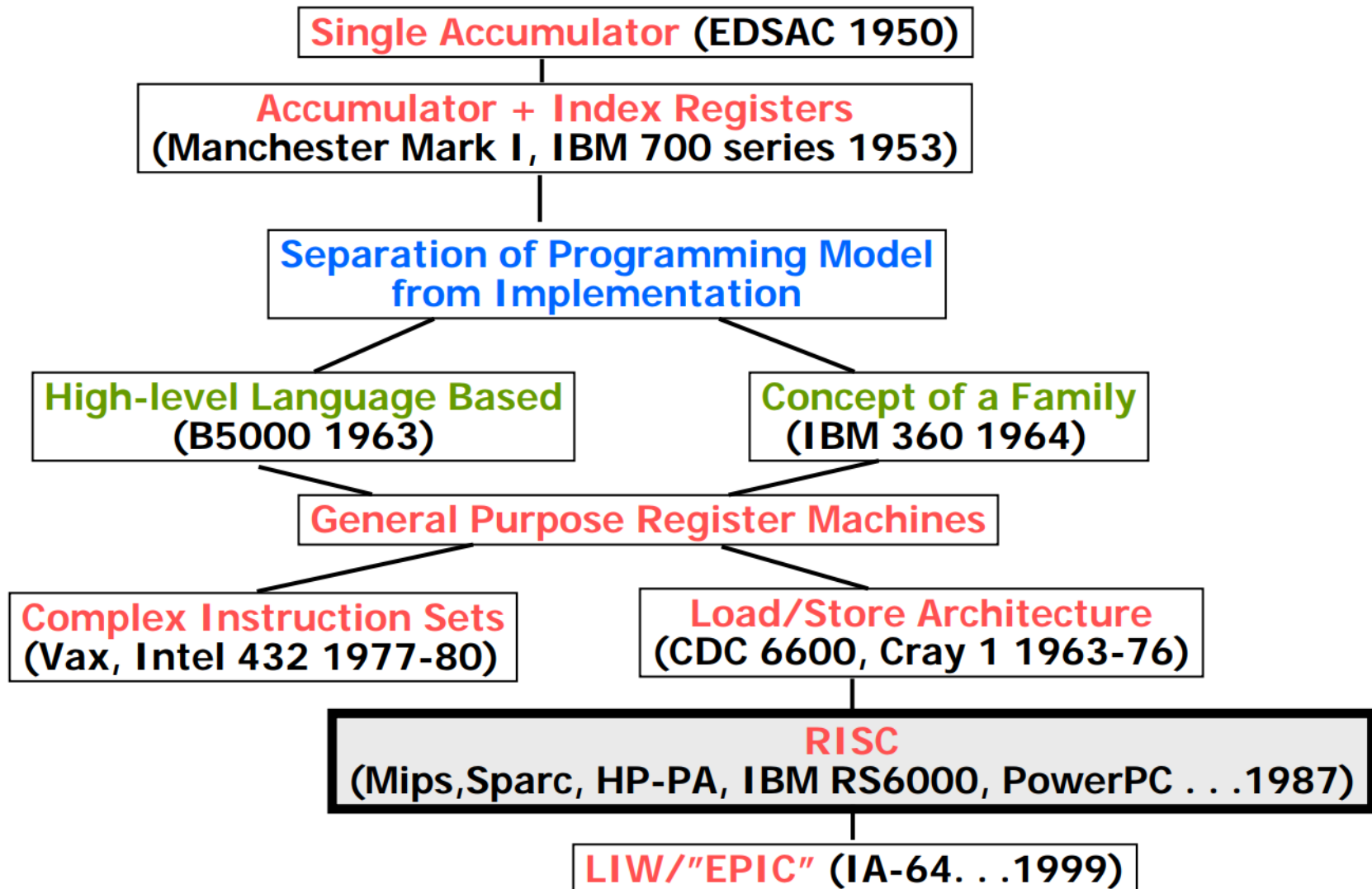
Memory mapped I/O (Example)



Basic ISA classes

- **Stack** machines (e.g. HP3000):
 - Implicit operands (top of stack)
- **Accumulator** machines (e.g. PDP-8)
 - 1 explicit operand, **destination = accumulator**
- **Register** machines: **1 or 2 explicit operands**, and a **destination**:
 - **Memory operands** (register-memory/memory-memory) (e.g. x-86)
 - **Load-store** (e.g. MIPS):
 - Register-register
 - **No memory** operands for ALU instructions
 - Only special **data-movement** can have **memory operands**

Evolution of ISA



Computer Architecture Taxonomy

- Two main approaches to speeding up execution through instruction design
- The complexity of instructions and how they are executed:
 - CISC: complex instruction set computers
 - RISC: reduced instruction set computers

Instruction Set Architectures

- Early trend was to **add more and more instructions to new CPUs** to do elaborate operations
 - **VAX architecture** had an instruction to **multiply polynomials!**
- RISC philosophy (IBM, Patterson, Hennessy, 1980s) (**Reduced Instruction Set Computing**)
 - Keep the instruction set **small** and **simple**, makes it **easier** to **build fast hardware**.
 - Let **software** do complicated operations by decomposing it to **simpler ones**.

RISC vs. CISC

Multiply Two Numbers (2:3, 5:2) in Memory:

CISC approach:

MULT 2:3, 5:2

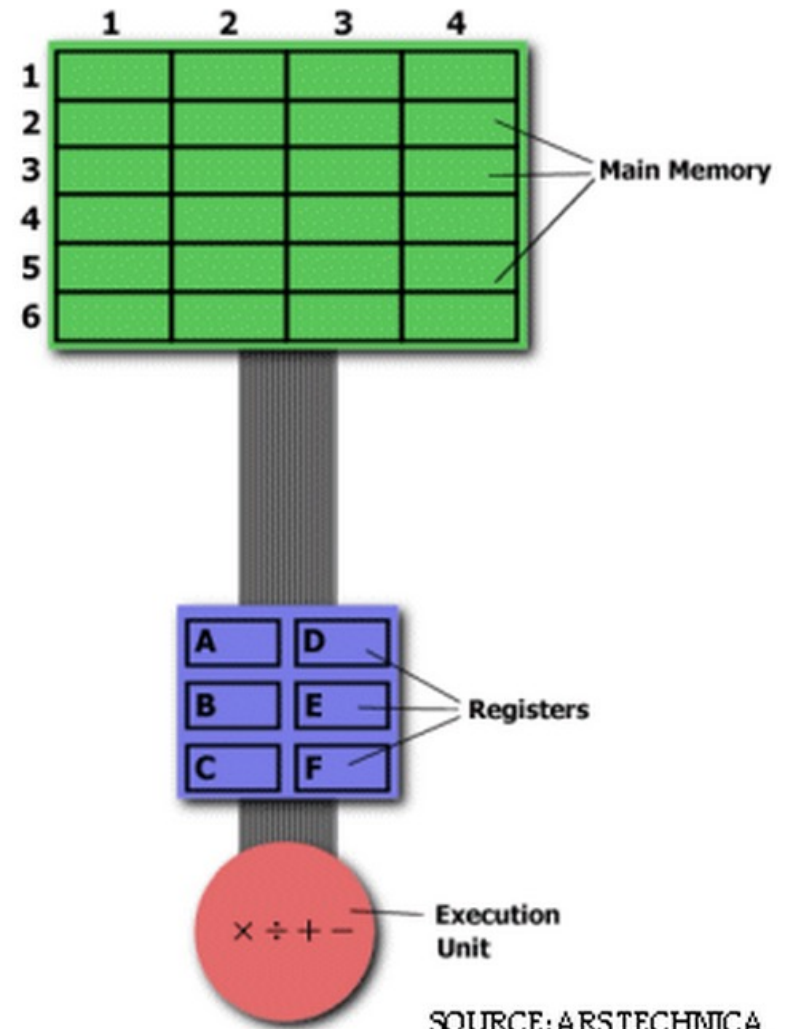
RISC approach:

LOAD A, 2:3

LOAD B, 5:2

PROD A, B

STORE 2:3, A



RISC vs. CISC

```
1 int square(int num)
2 {
3     return num * num;
4 }
```

C++ Code

```
1 square(int):
2     str fp, [sp, #-4]!
3     add fp, sp, #0
4     sub sp, sp, #12
5     str r0, [fp, #-8]
6     ldr r3, [fp, #-8]
7     ldr r2, [fp, #-8]
8     mul r1, r2, r3
9     mov r3, r1
10    mov r0, r3
11    add sp, fp, #0
12    ldr fp, [sp], #4
13    bx lr
```

ARM gcc 8.2 RISC

```
1 square(int):
2     push rbp
3     mov rbp, rsp
4     mov DWORD PTR [rbp-4], edi
5     mov eax, DWORD PTR [rbp-4]
6     imul eax, DWORD PTR [rbp-4]
7     pop rbp
8     ret
```

x86-64 gcc 8.2 CISC

RISC vs. CISC (cont.)

CISC	RISC
Emphasis on hardware	Emphasis on software
Includes multi-clock complex instructions	Single-clock, reduced instruction only
Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	Register to register: "LOAD" and "STORE" are independent instructions
Small code sizes, high cycles per instruction	Low cycles per instruction, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

$$\frac{\text{time}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{time}}{\text{cycle}}$$

CISC: minimizes the number of instructions per program at cost of number of cycles per instruction

RISC: reduces the cycles per instruction at the cost of the number of instructions per program

CISC approach

- Variable-length instructions that have many formats
 - Hard to implement fetch and decode
 - Code is dense (less memory)
- Memory-Memory and Register-Memory architecture
 - Suffers from slow implementation (complex instructions)
 - Harder to pipeline
 - Code density
 - Less instructions than Load-Store ISAs
- Supports many addressing modes
 - Complex effective address (EA) calculation slows memory access
- Many complex arithmetic functions
 - More transistors → More power consumption (heat)

RISC approach

- Fixed-length instructions that have only a few formats
 - Simplify instruction fetch and decode
 - Sacrifice code density
 - Some bits are wasted for some instruction types
 - Requires more memory
- Load-store architecture
 - Allows fast implementation of simple instructions
 - Easier to pipeline
 - Code density
 - More instructions than register-memory and memory-memory ISAs
- Limited number of addressing modes
 - Simplify effective address (EA) calculation to speed up memory access
- Few (if any) complex arithmetic functions
 - Build these from simpler instructions

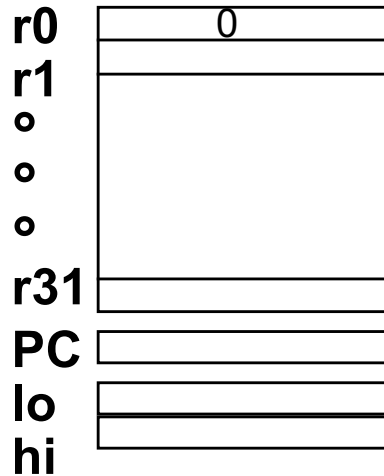
Semantic Gap

- Where to place the ISA? Semantic gap
 - Closer to high-level language (HLL) → Small semantic gap, complex instructions
 - Closer to hardware control signals? → Large semantic gap, simple instructions
- RISC vs. CISC machines ?
 - RISC: Reduced instruction set computer
 - CISC: Complex instruction set computer
 - FFT, QUICKSORT, POLY, FP instructions?
 - VAX INDEX instruction (array access with bounds checking)
 - Simple compiler, complex hardware vs. complex compiler, simple hardware

MIPS: A "Typical" RISC ISA

- 32-bit fixed format instruction (3 formats)
- Registers
 - 32 32-bit integer GPRs (R1-R31, R0 always = 0)
 - 32 32-bit floating-point GPRs (F0-F31)
 - For double-precision FP, registers paired
- 3-address, reg-reg arithmetic instruction
- Single address mode for load/store:
base + displacement
- Simple branch conditions
- Delayed branch

More MIPS



Programmable storage

2^{32} x bytes

31 x 32-bit GPRs (R0=0)

32 x 32-bit FP regs (paired DP)

HI, LO, PC

Arithmetic/ logical

Integer: DADD, DADDU, DSUB, DSUBU, AND, OR, XOR, SLT, SLTU, DMUL, DDIV

Immediate: DADDI, DADDIU, ANDI, ORI, XORI, SLTI, SLTIU

Shifts: DSLL, DSRL, DSRA

Floating point: ADD.D, ADD.S, SUB.D, SUB.S, MUL.D, MUL.S, DIV.D, DIV.S

Memory Access

Loads: LB, LBU, LH, LHU, LW, L.S, L.D

Stores: SB, SH, SW, S.S, S.D

Control

Jumps (unconditional): J, JAL, JR, JALR

Branches (conditional): BEQ, BNE, BEQZ, BNEZ (+ pseudo-instructions)

Reading

- x86 Architecture Overview
<https://cs.lmu.edu/~ray/notes/x86overview/>
- Appendix k from the textbook
- Review MIPS ISA and Assembly Language
- Optional:
 - RISC-V: <https://en.wikipedia.org/wiki/RISC-V>
 - ARM (Advanced RISC Machines):
https://en.wikipedia.org/wiki/ARM_architecture_family